

Πολυτεχνείο Κρήτης

Διπλωματική Εργασία

**Μια Ολοκληρωμένη, Έξυπνη,
Πλήρως-Διαμορφώσιμη Πλατφόρμα για
Επιχειρήσεις Διαχείρισης Ακινήτων**

Συγγραφέας:

Θεόδωρος Μπάρκας

Εξεταστική Επιτροπή:

Καθ. Μιχαήλ Γ. Λαγουδάκης

Καθ. Αντώνιος Δεληγιαννάκης

Αναπλ. Καθ. Βασίλειος

Σαμολαδάς



Διπλωματική εργασία που υποβλήθηκε για την απόκτηση
του διπλώματος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών
στην

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εργαστήριο Μικροεπεξεργαστών και Υλικού

Χανιά, Νοέμβριος 2025

ΠΝΕΥΜΑΤΙΚΑ ΔΙΚΑΙΩΜΑΤΑ

«Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για μη κερδοσκοπικό σκοπό, εκπαιδευτικού ή ερευνητικού χαρακτήρα, με την προϋπόθεση να αναφέρεται η πηγή προέλευσης. Ερωτήματα που αφορούν τη χρήση της εργασίας για άλλη χρήση θα πρέπει να απευθύνονται προς τον συγγραφέα.»

«Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Πολυτεχνείου Κρήτης.»

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

Περίληψη

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών

**Μια Ολοκληρωμένη, Έξυπνη, Πλήρως-Διαμορφώσιμη Πλατφόρμα για
Επιχειρήσεις Διαχείρισης Ακινήτων**

by Θεόδωρος Μπάρκας

Η παρούσα διπλωματική εργασία πραγματεύεται τον σχεδιασμό και την υλοποίηση της πλατφόρμας «Lunarety», ενός προηγμένου συστήματος διαχείρισης τουριστικών καταλυμάτων (Property Management System -- PMS) που αξιοποιεί σύγχρονες τεχνολογίες για να προσφέρει λύσεις Rapid Development και ενσωμάτωση Τεχνητής Νοημοσύνης (LLMs). Σκοπός της εργασίας είναι η κάλυψη του κενού που υπάρχει στην αγορά για ένα εργαλείο που απευθύνεται σε διαχειριστές ακινήτων (Managers) οι οποίοι επιβλέπουν πολλαπλούς ιδιοκτήτες (Owners), προσφέροντας διαφάνεια, αυτοματισμό και ευκολία χρήσης που λείπει από τα παραδοσιακά Channel Managers.

Η μεθοδολογία ανάπτυξης βασίστηκε στη χρήση του PayloadCMS v3.0, το οποίο λειτουργεί ως ένα πλήρες backend framework εντός του Next.js, προσφέροντας Code-First ορισμό δομών δεδομένων και ισχυρή τυποποίηση μέσω TypeScript. Η βάση δεδομένων υλοποιήθηκε σε PostgreSQL με χρήση Drizzle ORM, εξασφαλίζοντας ακεραιότητα και ταχύτητα. Η πλατφόρμα ακολουθεί αρχιτεκτονική Monorepo και είναι σχεδιασμένη για Serverless Deployment (π.χ. Vercel), επιτρέποντας στους διαχειριστές να ξεκινούν με μηδενικό κόστος υποδομής και να κλιμακώνονται ανάλογα με τις ανάγκες τους.

Κεντρικό στοιχείο της καινοτομίας αποτελεί η ενσωμάτωση Large Language Models (LLMs) μέσω του Vercel AI SDK και του OpenRouter, προσφέροντας έξυπνες λειτουργίες σε πολλαπλά επίπεδα. Επιπλέον, αναπτύχθηκε ένας μηχανισμός «Auto Actions» για την πλήρη αυτοματοποίηση της επικοινωνίας μέσω πολλαπλών καναλιών (Email, OTA Messages, Telegram).

Η εργασία συνεισφέρει στην κατανόηση του τρόπου με τον οποίο οι σύγχρονες αρχιτεκτονικές λογισμικού και τα εργαλεία Rapid Development μπορούν να μετασχηματίσουν επιχειρησιακές διαδικασίες, προσφέροντας λύσεις υψηλής αξίας με μειωμένο χρόνο ανάπτυξης και συντήρησης.

Λέξεις-κλειδιά: Property Management System, Rapid Development, Large Language Models, PayloadCMS, Next.js, TypeScript, Serverless, Channel Manager, Auto Actions

TECHNICAL UNIVERSITY OF CRETE

Abstract

School of Electrical and Computer Engineering

Electrical and Computer Engineer

**A Comprehensive, Smart, Fully-Configurable Platform for Property
Management Businesses**

by Theodoros Barkas

This diploma thesis deals with the design and implementation of the «Lunarety» platform, an advanced Property Management System (PMS) that leverages modern technologies to offer Rapid Development solutions and Large Language Model (LLM) integration. The purpose of this work is to bridge the market gap for a tool tailored to Property Managers overseeing multiple Owners, offering transparency, automation, and ease of use often missing from traditional Channel Managers.

The development methodology was based on PayloadCMS v3.0, acting as a full backend framework within Next.js, offering Code-First data structure definitions and strong typing via TypeScript. The database was implemented in PostgreSQL using Drizzle ORM, ensuring data integrity and performance. The platform follows a Monorepo architecture and is designed for Serverless Deployment (e.g., Vercel), allowing managers to start with zero infrastructure costs and scale as needed.

A key innovation element is the integration of Large Language Models (LLMs) via Vercel AI SDK and OpenRouter, offering smart features at multiple levels. Furthermore, an «Auto Actions» mechanism was developed to fully automate communication through multiple channels (Email, OTA Messages, Telegram).

This work contributes to understanding how modern software architectures and Rapid Development tools can transform business processes, delivering high-value solutions with reduced development and maintenance time.

Keywords: Property Management System, Rapid Development, Large Language Models, PayloadCMS, Next.js, TypeScript, Serverless, Channel Manager, Auto Actions

Acknowledgements

Θα ήθελα να ευχαριστήσω την οικογένειά μου για την αμέριστη υποστήριξή τους καθ' όλη τη διάρκεια των σπουδών μου και της εκπόνησης αυτής της διπλωματικής εργασίας.

Επίσης, θα ήθελα να εκφράσω τις ειλικρινείς μου ευχαριστίες στον επιβλέποντα καθηγητή μου, Καθ. Μιχαήλ Γ. Λαγουδάκη, για την καθοδήγηση, τις πολύτιμες συμβουλές και την εμπιστοσύνη που μου έδειξε κατά τη διάρκεια αυτής της εργασίας.

Contents

Περίληψη	iii
Abstract	iv
Acknowledgements	vi
Contents	vii
List of Figures	xi
List of Tables	xiii
List of Abbreviations	xiv
1 Εισαγωγή	1
1.1 Αντικείμενο της Μελέτης	1
1.2 Το Κενό στην Αγορά: Managers και Owners	2
1.2.1 Αρχιτεκτονική Channel Adapter	2
1.3 Σκοπός της Διπλωματικής Εργασίας	3
1.3.1 Σχεδιασμός για Ευχρηστία και Προσβασιμότητα	3
1.4 Συνεισφορά της Πλατφόρμας	4
1.5 Δομή της Εργασίας	5
2 Θεωρητικό Υπόβαθρο	6
2.1 Σύγχρονες Αρχιτεκτονικές Web	6
2.1.1 TypeScript και Type Consistency	7
2.1.2 Serverless Architecture	7
2.1.3 Next.js: Ένα Full-Stack React Framework	8
2.1.4 PayloadCMS: Ένα Modern Backend Framework	9
CollectionConfig: Ο Ενιαίος Ορισμός	9
Lifecycle Hooks και Direct Database Access	12
Drizzle ORM: Type-Safe Database Access	13
2.1.5 PostgreSQL: Σχεσιακή Βάση Δεδομένων	13

2.1.6	Shaden UI, React και Zustand	13
2.1.7	Swagger/OpenAPI και Code Generation	14
2.2	Large Language Models (LLMs)	14
2.2.1	Vercel AI SDK και Streaming	15
2.2.2	OpenRouter: Ενοποιημένη Πρόσβαση σε LLM Μοντέλα	15
2.3	Channel Managers και Διασυνδεσιμότητα	16
2.3.1	Τι είναι ο Channel Manager	16
2.3.2	Γιατί είναι Απαραίτητοι οι Channel Managers	17
2.3.3	Beds24: Ένας Δημοφιλής Channel Manager	18
3	Μεθοδολογία και Ανάλυση Απαιτήσεων	19
3.1	Ρόλοι και Ιεραρχία Χρηστών	19
3.1.1	Τύποι Χρηστών (User Types)	19
3.1.2	Ιεραρχικές Σχέσεις και Billing	21
3.2	Αρχιτεκτονική Συστήματος (Monorepo)	21
3.2.1	Δομή Monorepo	22
	Platform Repository (lunarety)	22
	Website Template Repository (lunarety-website)	22
3.2.2	Βασικά Components	23
3.2.3	External Integrations	24
3.3	Μοντέλο Δεδομένων και Σχέσεις	25
3.3.1	Κύριες Οντότητες (Collections)	25
3.3.2	Σχέσεις Μεταξύ Οντοτήτων	26
3.3.3	Hook System και Lifecycle	26
4	Υλοποίηση	28
4.1	Αρχιτεκτονική Lifecycle Hooks Μεταξύ Collections	28
4.1.1	Βασικά Αρχιτεκτονικά Patterns	29
4.2	Διαχείριση Κρατήσεων και Συγχρονισμός	30
4.2.1	Αρχική Σύνδεση και Ρύθμιση Webhooks	31
4.2.2	Μέθοδοι Εισαγωγής Κρατήσεων	31
	A) Mass Import (Property Syncing)	32
	B) Webhook Updates (Channel Manager Triggers)	33
	Γ) UI-Based Booking Creation	34
	Δ) Website Booking Creation	35
4.2.3	Σύστημα Σύνδεσης με Property Rates	35
4.2.4	Διαχείριση Transactions και Ασφάλεια Δεδομένων	37
4.2.5	Εισαγωγή Properties και Βελτιστοποίηση Property Rates	37
4.3	Μηχανισμός Auto Actions	40

4.3.1	Κατηγορίες Auto Actions	40
4.3.2	Κανάλια Επικοινωνίας	40
4.3.3	Μηχανισμός Προτύπων (Template Engine)	41
4.3.4	Time-based Triggers (Cron Jobs)	42
4.3.5	Instant Triggers (Webhooks)	44
4.3.6	Αρχιτεκτονική Cron Jobs	45
4.4	Σύστημα Χρέωσης και Τιμολόγησης	45
4.4.1	Αρχιτεκτονική Χρέωσης	45
4.4.2	Αυτόματη Μηνιαία Δημιουργία Bills (Cron Job)	45
4.4.3	Ανάλυση Χρεώσεων (Pricing Breakdown)	46
4.4.4	Προβολή Bills ανά Ρόλο Χρήστη	46
4.4.5	Κατάσταση Πληρωμής	46
4.4.6	Σύστημα Ελέγχου Πρόσβασης (Access Control System)	47
4.5	Αυτοματοποιημένη Παραγωγή Ιστοσελίδων	49
4.5.1	Δομή και Λειτουργία Website	49
4.5.2	Website API Endpoints	50
4.5.3	Deploy URL & Vercel Integration	52
4.5.4	Website Booking Creation Flow	52
4.5.5	Επιχειρηματικές Ευκαιρίες (Platform as OTA)	53
4.6	Περιβάλλον Χρήστη (Platform UI)	54
4.6.1	Dashboard και Calendar	54
4.6.2	Σύστημα Μηνυμάτων	54
4.6.3	Διαχείριση Δικαιωμάτων και Drawers	55
5	Αποτελέσματα	56
5.1	Παρουσίαση της Πλατφόρμας	56
5.2	Παρουσίαση του Generated Website	58
5.3	Smart Features & LLM Integration	60
5.4	Τεχνικές Προκλήσεις και Λύσεις	61
5.4.1	Συγχρονισμός Δεδομένων με External APIs	61
5.4.2	Αποφυγή Infinite Loops	61
5.4.3	Απόδοση Μαζικών Εισαγωγών	61
5.4.4	Διαχείριση Σύνθετων Rate Structures	62
5.4.5	Multi-Tenant Data Isolation	63
5.4.6	Time-Zone Management	64
5.5	Στατιστικά και Μετρήσεις Κώδικα	65
6	Συζήτηση	66
6.1	Αξιολόγηση Τεχνολογικών Επιλογών	66

6.2	Μελλοντικές Βελτιώσεις	67
6.2.1	Βελτίωση UX στο Admin Panel	67
6.2.2	Επέκταση Channel Manager Support	67
6.2.3	Mobile Application	67
6.2.4	Advanced Analytics Dashboard	67
6.2.5	Payment Integration	68
7	Συμπεράσματα	69
7.1	Επίτευξη Στόχων	69
7.2	Τεχνικά Συμπεράσματα	69
7.3	Επιχειρηματικό Αντίκτυπο	70
7.4	Βασικά Διδάγματα	70
	Βιβλιογραφία	72
A	Δομή Webhook Payload	74
B	Παράδειγμα Auto Action Query	75

List of Figures

2.1	Σύγχρονο τεχνολογικό stack για web εφαρμογές (Next.js, Payload-CMS, PostgreSQL, κ.α.).	6
2.2	Αρχιτεκτονική PayloadCMS: Ένας ενιαίος ορισμός δημιουργεί πολλαπλά επίπεδα λειτουργικότητας.	9
2.3	Ροή παραγωγής Client από OpenAPI/Swagger Documentation.	14
2.4	Αρχιτεκτονική OpenRouter: Ένας ενδιάμεσος κόμβος για πρόσβαση σε πληθώρα LLM μοντέλων.	16
2.5	Αρχιτεκτονική Channel Manager ως ενδιάμεσου κόμβου.	17
3.1	Ιεραρχική δομή ρόλων χρηστών στην πλατφόρμα.	21
3.2	Διάγραμμα αρχιτεκτονικής συστήματος (Monorepo).	24
3.3	Entity Relationship Diagram (ERD) του μοντέλου δεδομένων της πλατφόρμας.	27
4.1	Διάγραμμα ροής lifecycle hooks μεταξύ των κύριων Collections της πλατφόρμας. Κάθε collection ενεργοποιεί συγκεκριμένες ροές βάσει συνθηκών, με τελικό σημείο σύγκλισης τα PropertyRates.	29
4.2	Μέθοδοι εισαγωγής κρατήσεων στην πλατφόρμα.	32
4.3	Τριπλή δομή προμηθειών (Three-Tier Commission Structure).	34
4.4	Ροή δεδομένων συγχρονισμού κρατήσεων με το Beds24.	35
4.5	Σύγκριση απόδοσης εισαγωγής rates.	40
4.6	Διάγραμμα ροής εκτέλεσης των Auto Actions.	41
4.7	Σενάρια λειτουργίας Time-based Triggers για κράτηση της τελευταίας στιγμής (1 μέρα πριν την άφιξη). Στο Σενάριο 1 (μικρό παράθυρο 5 ωρών), η κράτηση χάνει τον κανόνα «2 μέρες πριν» και δεν στέλνεται μήνυμα. Στο Σενάριο 2 (μεγάλο παράθυρο 2 ημερών), η κράτηση εμπίπτει στο παράθυρο και το μήνυμα (π.χ. «The key to open your door is : 2004») αποστέλλεται κανονικά.	44
4.8	Διεπαφή χρήστη Billing.	47
4.9	Decision tree για τον έλεγχο πρόσβασης.	49
4.10	Αρχιτεκτονική σελίδων Website.	50
4.11	Ροή επικοινωνίας Website API.	51

4.12	Διαδικασία One-Click Deploy.	52
4.13	Ταξίδι κράτησης επισκέπτη.	53
4.14	Επιχειρηματικό μοντέλο Platform as OTA.	54
5.1	Η σελίδα του Ημερολογίου (Calendar). Το αρχικό ημερολόγιο έχει σχεδιαστεί ως ένα «Easy Calendar». Για τον βασικό χρήστη, λειτουργεί ως μια απλή λίστα καθηκόντων που δείχνει τι πρέπει να γίνει κάθε μέρα (αφίξεις, αναχωρήσεις), καθιστώντας το εξαιρετικά εύχρηστο για την καθημερινότητα. Ωστόσο, προσφέρει τη δυνατότητα προσθήκης περισσότερης λειτουργικότητας για πιο απαιτητικούς χρήστες, χωρίς να θυσιάζεται η ευκολία χρήσης.	56
5.2	Η σελίδα των Μηνυμάτων. Το σύστημα μηνυμάτων παρέχει δυνατότητες άμεσης επικοινωνίας, υποστηρίζοντας τη χρήση προτύπων (templates) και τη βελτίωση κειμένου μέσω AI.	57
5.3	Διαχείριση Χρηστών (Users) στο Advanced Dashboard.	57
5.4	Διαχείριση Πλάνων Χρέωσης (Plans) στο Advanced Dashboard.	58
5.5	Διαχείριση Αυτοματισμών (Auto Actions) στο Advanced Dashboard.	58
5.6	Διαδικασία παραγωγής Website από το περιβάλλον του διαχειριστή. Ο διαχειριστής μπορεί με απλά βήματα να δημιουργήσει και να δημοσιεύσει την ιστοσελίδα του καταλύματος, αξιοποιώντας τα δεδομένα που είναι ήδη καταχωρημένα στην πλατφόρμα.	59
5.7	Πλοήγηση επισκέπτη στην παραγόμενη ιστοσελίδα. Η ιστοσελίδα που παράγεται είναι πλήρως λειτουργική, επιτρέποντας στον επισκέπτη να δει λεπτομέρειες του καταλύματος, φωτογραφίες και παροχές, καθώς και να προχωρήσει σε κράτηση.	59
5.8	Δυνατότητες AI και LLM για την εξυπηρέτηση επισκεπτών. Ενσωματωμένες δυνατότητες Τεχνητής Νοημοσύνης επιτρέπουν στους επισκέπτες να αλληλεπιδρούν με έξυπνους βοηθούς για να λαμβάνουν άμεσες απαντήσεις σε ερωτήματα σχετικά με το κατάλυμα και την περιοχή.	60
5.9	Σύγκριση απόδοσης Import.	61
5.10	Μετατροπή δομής Rate.	62
5.11	Απομόνωση δεδομένων Multi-Tenant.	63
5.12	Διαχείριση ζωνών ώρας.	64
5.13	Στατιστικά Κώδικα.	65

List of Tables

3.1	Τύποι χρηστών της πλατφόρμας	20
3.2	External Services και Integrations	24
3.3	Κύριες Collections της βάσης δεδομένων	25
3.4	Hook Types και χρήση στην πλατφόρμα	26
4.1	Στρατηγικές Βελτιστοποίησης Rates	39
4.2	Υποστηριζόμενοι Time-based Trigger Types	43
4.3	Scheduled Cron Jobs της πλατφόρμας	45
4.4	Τύποι χρεώσεων Billing	46
4.5	Προβολή Billing ανά Ρόλο	46
4.6	Πίνακας Δικαιωμάτων	47
4.7	Website API Endpoints	51
5.1	Σύγκριση στρατηγικών εισαγωγής	61
5.2	Στατιστικά project	65
6.1	Αξιολόγηση τεχνολογικών επιλογών	67
7.1	Κύρια επιτεύγματα της πλατφόρμας	69

List of Abbreviations

PMS	Property Management System
LLM	Large Language Model
CMS	Content Management System
API	Application Programming Interface
OTA	Online Travel Agency
ORM	Object Relational Mapping
SDK	Software Development Kit
SSR	Server Side Rendering
SSG	Static Site Generation
RAD	Rapid Application Development
UI	User Interface
UX	User Experience
ERD	Entity Relationship Diagram
DTO	Data Transfer Object
ACID	Atomicity Consistency Isolation Durability

Αφιερωμένο στην οικογένεια και τους φίλους μου...

Chapter 1

Εισαγωγή

1.1 Αντικείμενο της Μελέτης

Το αντικείμενο της παρούσας διπλωματικής εργασίας είναι ο σχεδιασμός και η ανάπτυξη ενός ολοκληρωμένου πληροφοριακού συστήματος, του «Lunarety», για τη διαχείριση τουριστικών καταλυμάτων. Η πλατφόρμα δεν αποτελεί απλώς ένα ακόμη PMS, αλλά μια ολιστική λύση που στοχεύει στην κεντρικοποίηση των λειτουργιών που απαιτούνται από επαγγελματίες διαχειριστές (Managers) και τους πελάτες τους, τους ιδιοκτήτες ακινήτων (Owners). Πέρα από τις βασικές λειτουργίες διαχείρισης κρατήσεων και συγχρονισμού ημερολογίων, η πλατφόρμα ενσωματώνει προηγμένες δυνατότητες Τεχνητής Νοημοσύνης (LLMs) για την υποβοήθηση της επικοινωνίας και την παροχή έξυπνων υπηρεσιών.

Κεντρική φιλοσοφία της πλατφόρμας είναι η **διαχείριση της πολυπλοκότητας ανά ρόλο χρήστη**. Κάθε χρήστης—είτε πρόκειται για Manager, Owner, είτε για απλό προσωπικό (Staff)—αντικρίζει ένα απλό και κατανοητό περιβάλλον εργασίας που περιλαμβάνει μόνο τις πληροφορίες και τις λειτουργίες που αφορούν τον δικό του ρόλο και τις δικές του ευθύνες. Η πολυπλοκότητα του συστήματος παραμένει κρυμμένη από τους χρήστες που δεν χρειάζεται να την γνωρίζουν, ενώ αποκαλύπτεται σταδιακά σε αυτούς που την χρειάζονται. Αυτή η προσέγγιση εξασφαλίζει ότι κάθε χρήστης έχει πλήρη εικόνα για τη δική του εργασία και λογοδοσία, χωρίς να επιβαρύνεται με περιττές πληροφορίες—ένα χαρακτηριστικό που απουσιάζει από τα περισσότερα υπάρχοντα συστήματα PMS.

Επιπλέον, η πλατφόρμα αναγνωρίζει τον **Property Manager ως εντολοδόχο που δικαιούται προμήθειας**. Τα περισσότερα συστήματα PMS αντιμετωπίζουν τον ιδιοκτήτη ως κάποιον που πληρώνει προμήθεια μόνο στα OTAs (Booking.com, Airbnb), αγνοώντας τη σχέση Manager-Owner. Αυτό αναγκάζει τους Managers να επεξεργάζονται χειροκίνητα τα τιμολόγια και να τα αποστέλλουν στους πελάτες τους. Το Lunarety ενσωματώνει

αυτή τη σχέση στον πυρήνα του, αυτοματοποιώντας τον υπολογισμό και την τιμολόγηση των προμηθειών μεταξύ όλων των εμπλεκόμενων μερών.

Τέλος, η πλατφόρμα διαθέτει ένα **σύγχρονο και διαισθητικό περιβάλλον χρήστη**, σχεδιασμένο να λειτουργεί άψογα τόσο σε desktop όσο και σε κινητές συσκευές. Η διεπαφή ακολουθεί καθιερωμένα πρότυπα UX που είναι οικεία στους χρήστες από εφαρμογές καθημερινής χρήσης, μειώνοντας δραστικά τον χρόνο εκμάθησης ακόμη και για χρήστες που δυσκολεύονται να υιοθετήσουν νέες τεχνολογίες.

1.2 Το Κενό στην Αγορά: Managers και Owners

Η παρούσα εργασία απευθύνεται κυρίως σε επαγγελματίες Property Managers που διαχειρίζονται χαρτοφυλάκια ακινήτων που ανήκουν σε τρίτους (Owners). Στην τρέχουσα αγορά, παρατηρείται ένα σημαντικό κενό:

- Οι Managers συχνά αναγκάζονται να χρησιμοποιούν πολύπλοκα λογισμικά Channel Managers, τα οποία είναι δύσχρηστα για τους Owners ή το απλό προσωπικό.
- Η ενημέρωση των Owners για κρατήσεις, πληρότητες και οικονομικά στοιχεία γίνεται συχνά χειροκίνητα (τηλέφωνα, excel, emails), οδηγώντας σε λάθη και καθυστερήσεις.
- Η δημιουργία λογαριασμών για Owners μέσα στα υπάρχοντα Channel Managers είναι συχνά δαπανηρή ή πολύπλοκη διαδικασία.
- Τα υπάρχοντα συστήματα δεν αναγνωρίζουν τον Manager ως εντολοδόχο με δικαίωμα προμήθειας, αναγκάζοντάς τον να διαχειρίζεται χειροκίνητα τη χρέωση και τιμολόγηση των πελατών του.

Το Lunarety έρχεται να καλύψει αυτό το κενό, προσφέροντας μια ενδιάμεση, φιλική προς τον χρήστη πλατφόρμα, όπου ο Manager ορίζει τα δικαιώματα, τις συνδρομές και την πρόσβαση των Owners, παρέχοντάς τους ακριβώς την πληροφορία που χρειάζονται σε πραγματικό χρόνο.

1.2.1 Αρχιτεκτονική Channel Adapter

Αντί να αντικαταστήσει τους υπάρχοντες Channel Managers, η πλατφόρμα σχεδιάστηκε ώστε να **λειτουργεί επιπρόσθετα σε αυτούς** μέσω μιας αρχιτεκτονικής Channel Adapters. Αυτή η προσέγγιση επιτρέπει στους Managers να συνεχίσουν να χρησιμοποιούν τα υπάρχοντα εργαλεία τους (π.χ. Beds24) για τον συγχρονισμό με τα OTAs, ενώ παράλληλα επεκτείνουν τη λειτουργικότητά τους με τα χαρακτηριστικά του Lunarety.

Επί του παρόντος, η πλατφόρμα υποστηρίζει ένα Channel Manager (Beds24), αλλά η αρχιτεκτονική έχει σχεδιαστεί ώστε να επιτρέπει την εύκολη προσθήκη νέων Adapters για επιπλέον Channel Managers στο μέλλον. Κάθε Adapter μεταφράζει τις κλήσεις της πλατφόρμας στο αντίστοιχο API του Channel Manager, διατηρώντας τον υπόλοιπο κώδικα της εφαρμογής ανεξάρτητο από τον συγκεκριμένο πάροχο.

1.3 Σκοπός της Διπλωματικής Εργασίας

Ο κύριος σκοπός της εργασίας είναι η δημιουργία ενός εργαλείου που μειώνει δραστικά τον διοικητικό φόρτο και εκσυγχρονίζει τις διαδικασίες διαχείρισης. Ειδικότερα, επιδιώκεται:

- Η δημιουργία ενός robust συστήματος διαχείρισης σχέσεων Manager-Owner, προσφέροντας διαφάνεια και αυτοματοποιημένη ενημέρωση.
- Η ενσωμάτωση δυνατοτήτων LLM (Large Language Models) τόσο για την υποβοήθηση των χρηστών στη σύνταξη μηνυμάτων όσο και για την παροχή smart features στους επισκέπτες μέσω των ιστοσελίδων των καταλυμάτων.
- Η υλοποίηση ενός ευέλικτου συστήματος «Auto Actions» τριών επιπέδων για την πλήρη αυτοματοποίηση των εργασιών.
- Η απρόσκοπτη διασύνδεση με τον Channel Manager «Beds24», με αρχιτεκτονική πρόβλεψη για μελλοντική προσθήκη άλλων παρόχων.
- Η ανάπτυξη ενός **σύγχρονου, φιλικού περιβάλλοντος χρήστη** που να είναι προσβάσιμο σε όλους, ανεξαρτήτως τεχνολογικής εξοικείωσης.

1.3.1 Σχεδιασμός για Ευχρηστία και Προσβασιμότητα

Ιδιαίτερη έμφαση δόθηκε στον σχεδιασμό του περιβάλλοντος χρήστη (UI/UX), με στόχο να νιώθει ο χρήστης «σαν στο σπίτι του» από την πρώτη στιγμή. Η διεπαφή ακολουθεί διαισθητικά πρότυπα σχεδιασμού που είναι ήδη γνωστά από δημοφιλείς εφαρμογές καθημερινής χρήσης:

- **Mobile-First Σχεδιασμός:** Η πλατφόρμα είναι πλήρως responsive, προσαρμοζόμενη αυτόματα σε κινητά τηλέφωνα και tablets. Οι περισσότεροι χρήστες (Owners, Staff) χρησιμοποιούν την εφαρμογή κυρίως από το κινητό τους.
- **Οικεία Πρότυπα Αλληλεπίδρασης:** Χρήση καθιερωμένων UI patterns όπως swipe gestures, pull-to-refresh, floating action buttons και bottom navigation bars—στοιχεία που οι χρήστες αναγνωρίζουν αμέσως από εφαρμογές όπως το WhatsApp ή το Instagram.

- **Απλοποιημένη Πλοήγηση:** Η δομή της εφαρμογής είναι επίπεδη και ξεκάθαρη. Οι βασικές λειτουργίες (Ημερολόγιο, Μηνύματα, Ρυθμίσεις) είναι προσβάσιμες με ένα κλικ.
- **Οπτική Σαφήνεια:** Μεγάλα κουμπιά, ευανάγνωστες γραμματοσειρές και υψηλή χρωματική αντίθεση διευκολύνουν τη χρήση ακόμη και σε δύσκολες συνθήκες φωτισμού ή από χρήστες μεγαλύτερης ηλικίας.

Αυτή η προσέγγιση εξασφαλίζει ότι ακόμη και χρήστες που παραδοσιακά δυσκολεύονται με τις νέες τεχνολογίες—όπως ιδιοκτήτες μεγαλύτερης ηλικίας ή προσωπικό χωρίς τεχνική κατάρτιση—μπορούν να αξιοποιήσουν πλήρως τις δυνατότητες της πλατφόρμας χωρίς εκτεταμένη εκπαίδευση.

1.4 Συνεισφορά της Πλατφόρμας

Η πλατφόρμα Lunarety συνεισφέρει ουσιαστικά στην καθημερινότητα όλων των εμπλεκομένων στη διαχείριση τουριστικών καταλυμάτων:

Για τους Property Managers: Παρέχει ένα κεντρικό εργαλείο διαχείρισης όλων των πελατών τους (Owners) με αυτοματοποιημένη τιμολόγηση, διαφανή υπολογισμό προμηθειών και πλήρη έλεγχο των δικαιωμάτων πρόσβασης. Ο Manager μπορεί να εστιάσει στην ανάπτυξη της επιχείρησής του αντί να χάνει χρόνο σε διοικητικές εργασίες.

Για τους Owners (Ιδιοκτήτες): Προσφέρει real-time ενημέρωση για τις κρατήσεις, τα έσοδα και τις υποχρεώσεις τους, χωρίς να απαιτείται τεχνική κατάρτιση. Κάθε Owner βλέπει μόνο τα δικά του ακίνητα σε ένα απλό και κατανοητό περιβάλλον.

Για το Προσωπικό (Staff): Επιτρέπει σε υπαλλήλους (καθαριστές, ρεσεψιονίστ) να έχουν πρόσβαση ακριβώς στις πληροφορίες που χρειάζονται για την εργασία τους—αφίξεις, αναχωρήσεις, καθήκοντα—χωρίς να τους επιβαρύνει με οικονομικά στοιχεία ή περίπλοκες ρυθμίσεις.

Για τους Επισκέπτες: Μέσω των αυτόματα παραγόμενων ιστοσελίδων με ενσωματωμένο AI assistant, οι επισκέπτες μπορούν να κάνουν απευθείας κρατήσεις και να λαμβάνουν άμεσες απαντήσεις στα ερωτήματά τους, 24 ώρες το 24ωρο.

Συνολικά, η πλατφόρμα μετατρέπει τη χαοτική διαχείριση πολλαπλών εργαλείων και χειροκίνητων διαδικασιών σε μια ενοποιημένη, αυτοματοποιημένη εμπειρία που εξυπηρετεί όλα τα επίπεδα του οικοσυστήματος της βραχυχρόνιας μίσθωσης.

1.5 Δομή της Εργασίας

Η παρούσα διπλωματική εργασία οργανώνεται στα ακόλουθα κεφάλαια:

Κεφάλαιο 2 – Θεωρητικό Υπόβαθρο: Παρουσιάζονται οι σύγχρονες αρχιτεκτονικές web, οι τεχνολογίες που χρησιμοποιήθηκαν (TypeScript, PayloadCMS, Next.js, PostgreSQL), καθώς και το θεωρητικό πλαίσιο των Large Language Models και των Channel Managers.

Κεφάλαιο 3 – Μεθοδολογία και Ανάλυση Απαιτήσεων: Αναλύεται η ιεραρχία χρηστών της πλατφόρμας, η αρχιτεκτονική του συστήματος (Monorepo), το μοντέλο δεδομένων και οι σχέσεις μεταξύ των οντοτήτων.

Κεφάλαιο 4 – Υλοποίηση: Περιγράφεται λεπτομερώς η υλοποίηση των βασικών λειτουργιών: διαχείριση κρατήσεων και συγχρονισμός με τον Channel Manager, μηχανισμός Auto Actions, σύστημα χρέωσης και τιμολόγησης, αρχιτεκτονική lifecycle hooks μεταξύ collections, έλεγχος πρόσβασης, και αυτοματοποιημένη παραγωγή ιστοσελίδων.

Κεφάλαιο 5 – Αποτελέσματα: Παρουσιάζεται η πλατφόρμα μέσω screenshots και GIFs, τα smart features με ενσωμάτωση LLM, καθώς και οι τεχνικές προκλήσεις που αντιμετωπίστηκαν μαζί με τις λύσεις τους.

Κεφάλαιο 6 – Συζήτηση: Αξιολογούνται οι τεχνολογικές επιλογές και προτείνονται μελλοντικές βελτιώσεις και επεκτάσεις της πλατφόρμας.

Κεφάλαιο 7 – Συμπεράσματα: Συνοψίζονται τα κύρια επιτεύγματα, τα τεχνικά συμπεράσματα, ο επιχειρηματικός αντίκτυπος και τα διδάγματα που αποκομίστηκαν από την ανάπτυξη της πλατφόρμας.

Chapter 2

Θεωρητικό Υπόβαθρο

2.1 Σύγχρονες Αρχιτεκτονικές Web

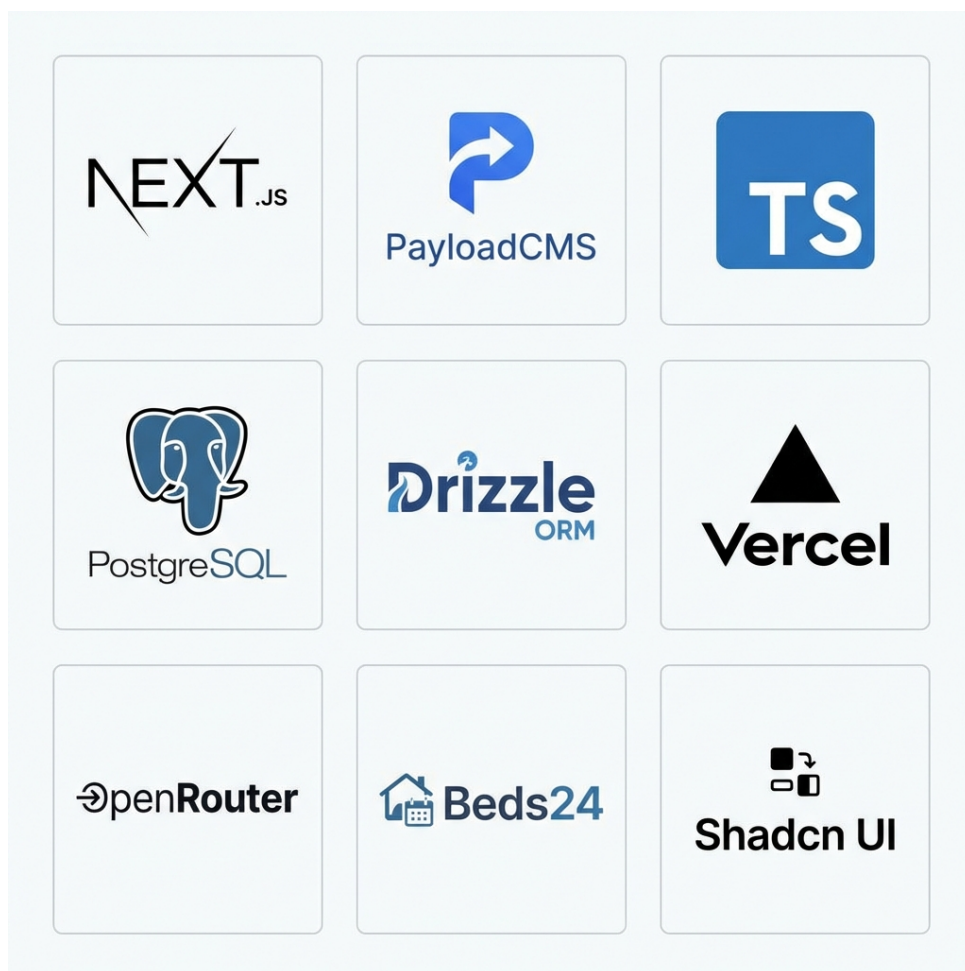


Figure 2.1: Σύγχρονο τεχνολογικό stack για web εφαρμογές (Next.js, PayloadCMS, PostgreSQL, κ.α.).

2.1.1 TypeScript και Type Consistency

Η TypeScript [1] είναι ένα superset της JavaScript που προσθέτει στατική τυποποίηση (static typing). Η στατική τυποποίηση εξασφαλίζει «Type Consistency» σε ολόκληρη την εφαρμογή (Fullstack), που σημαίνει ότι τα δεδομένα που επιστρέφει το Backend είναι εγγυημένα συμβατά με αυτά που περιμένει το Frontend. Αυτό μειώνει δραματικά τα runtime errors και επιταχύνει την ανάπτυξη μέσω του IntelliSense. Η TypeScript έχει καθιερωθεί ως το standard για την ανάπτυξη σύγχρονων, κλιμακώσιμων web εφαρμογών.

2.1.2 Serverless Architecture

Η Serverless Architecture [2] αποτελεί ένα μοντέλο cloud computing όπου ο πάροχος (π.χ. AWS, Vercel, Google Cloud) διαχειρίζεται αυτόματα την υποδομή των servers. Παρά το όνομά της, η serverless αρχιτεκτονική δεν σημαίνει απουσία servers – απλώς ο προγραμματιστής δεν χρειάζεται να τους διαχειρίζεται.

Βασικά Χαρακτηριστικά:

- **Auto-scaling:** Η υποδομή κλιμακώνεται αυτόματα ανάλογα με το φορτίο. Από μηδέν instances όταν δεν υπάρχει traffic, έως χιλιάδες όταν απαιτείται.
- **Pay-per-use:** Η χρέωση γίνεται βάσει πραγματικής χρήσης (αριθμός εκτελέσεων, χρόνος εκτέλεσης) αντί για σταθερό μηνιαίο κόστος server.
- **Zero DevOps:** Δεν απαιτείται διαχείριση servers, patches, ή υποδομής – ο προγραμματιστής εστιάζει μόνο στον κώδικα.
- **Event-driven:** Οι συναρτήσεις εκτελούνται ως απάντηση σε events (HTTP requests, database triggers, scheduled tasks).

Functions as a Service (FaaS): Το κυριότερο μοντέλο serverless είναι το FaaS, όπου ο κώδικας οργανώνεται σε μικρές, ανεξάρτητες συναρτήσεις. Κάθε συνάρτηση εκτελείται σε απομονωμένο περιβάλλον, ξεκινά όταν καλείται (cold start), και τερματίζεται μετά την ολοκλήρωση.

Πλεονεκτήματα και Περιορισμοί:

Πλεονεκτήματα: Μηδενικό αρχικό κόστος υποδομής, αυτόματη κλιμάκωση, μειωμένη πολυπλοκότητα deployment, ιδανικό για startups και MVPs.

Περιορισμοί: Cold starts (καθυστέρηση κατά την πρώτη κλήση), όρια χρόνου εκτέλεσης (συνήθως 10-60 δευτερόλεπτα), περιορισμένος έλεγχος υποδομής.

Πλατφόρμες όπως η **Vercel**, το **AWS Lambda**, και το **Google Cloud Functions** αποτελούν δημοφιλείς παρόχους serverless υπηρεσιών για web εφαρμογές.

2.1.3 Next.js: Ένα Full-Stack React Framework

Το Next.js [3] είναι ένα open-source React framework που αναπτύσσεται και συντηρείται από την Vercel. Προσφέρει δυνατότητες Server-Side Rendering (SSR) και Static Site Generation (SSG), καθιστώντας το ιδανικό για Serverless deployments [2]. Η αρχιτεκτονική του επιτρέπει σε frameworks όπως το PayloadCMS v3 να τρέχουν εντός του, δημιουργώντας ένα ενοποιημένο full-stack περιβάλλον.

Server Actions: Μια λειτουργία που επιτρέπει στο Frontend να καλεί συναρτήσεις του Backend απευθείας, χωρίς την ανάγκη ρητού ορισμού API Endpoints. Αυτό απλοποιεί τη διαχείριση δεδομένων και διατηρεί την τυποποίηση (types) από άκρη σε άκρη.

Monorepo Architecture: Η δομή του Next.js υποστηρίζει τη διατήρηση όλου του κώδικα (Frontend, Backend, Webhooks) σε ένα ενιαίο Repository, επιτρέποντας την κοινή χρήση τύπων και utilities και εξαλείφοντας την ανάγκη για συγχρονισμό μεταξύ διαφορετικών projects.

Local API: Όταν συνδυάζεται με CMS frameworks όπως το PayloadCMS, το Next.js υποστηρίζει `localAPI` – τυποποιημένη (typed) πρόσβαση στη βάση δεδομένων από οποιαδήποτε server function. Αυτό επιτρέπει κλήσεις όπως `payload.find()`, `payload.create()`, `payload.update()` με πλήρες autocomplete και type-checking, χωρίς HTTP requests.

2.1.4 PayloadCMS: Ένα Modern Backend Framework

Το PayloadCMS [4] (ειδικά από την έκδοση 3.0 και μετά) διαφοροποιείται σημαντικά από τα παραδοσιακά CMS (όπως το WordPress ή το Strapi). Λειτουργεί πρωτίστως ως ένα **Backend Framework** που τρέχει εντός του Next.js, παρόμοια με το Django ή το Laravel, αλλά με τη δύναμη του TypeScript. Η κατανόηση των δυνατοτήτων του είναι θεμελιώδης για την κατανόηση σύγχρονων αρχιτεκτονικών web εφαρμογών.

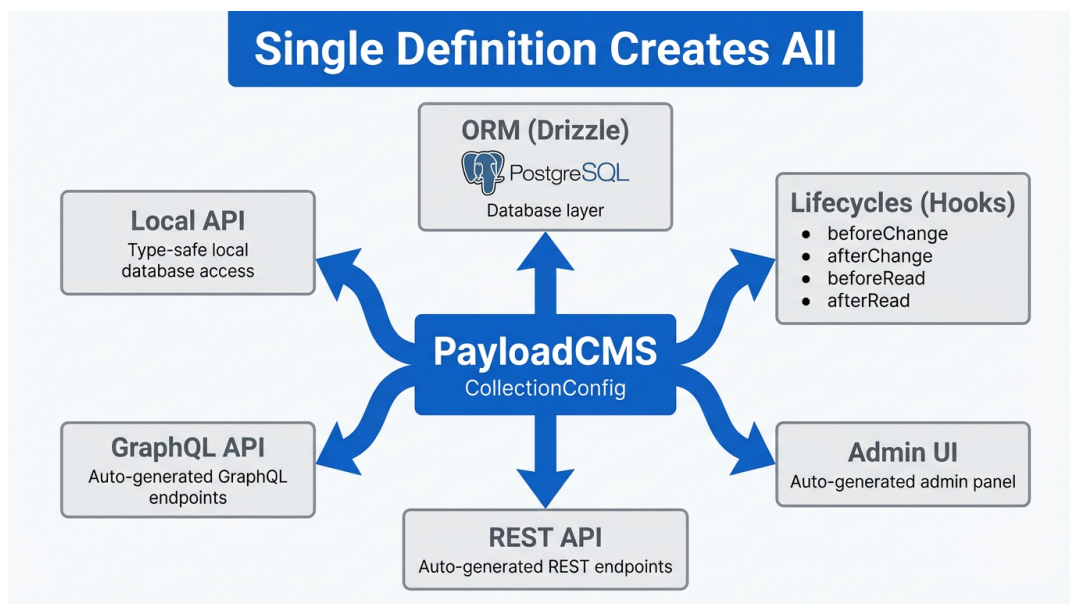


Figure 2.2: Αρχιτεκτονική PayloadCMS: Ένας ενιαίος ορισμός δημιουργεί πολλαπλά επίπεδα λειτουργικότητας.

CollectionConfig: Ο Ενιαίος Ορισμός

Το κεντρικό concept του PayloadCMS είναι το **CollectionConfig**. Ένα μόνο αντικείμενο TypeScript ορίζει ταυτόχρονα:

1. **Το σχήμα της βάσης δεδομένων (ORM):** Παρόμοια με τα Django Models ή τα Laravel Eloquent Models, τα fields ορίζουν τις στήλες του πίνακα.
2. **Το REST/GraphQL API:** Αυτόματη δημιουργία endpoints για CRUD operations.
3. **Τη διεπαφή διαχείρισης (Admin UI):** Ένα πλήρως λειτουργικό admin panel που παράγεται αυτόματα.
4. **Τη συμπεριφορά και τη λογική:** Hooks, access control, validation—όλα μέσα στο ίδιο config.

Το PayloadCMS υποστηρίζει πλούσιες σχέσεις μεταξύ collections μέσω του `relationship` field type. Υποστηρίζονται one-to-one, one-to-many, και many-to-many σχέσεις,

καθώς και polymorphic relations (ένα πεδίο που μπορεί να δείχνει σε πολλαπλά collections).

Το `join` field είναι ιδιαίτερα χρήσιμο καθώς δημιουργεί «virtual» πεδία που δεν αποθηκεύονται στη βάση αλλά υπολογίζονται δυναμικά. Η παράμετρος `where` επιτρέπει φιλτράρισμα των σχετιζόμενων εγγραφών.

Listing 2.1: Παράδειγμα CollectionConfig με custom components και conditional logic

```
1 import { CollectionConfig } from 'payload/types';
2 import { CustomPriceEditor } from '../components/CustomPriceEditor';
3
4 export const Bookings: CollectionConfig = {
5   slug: 'bookings',
6   // Access control
7   access: {
8     read: ({ req: { user } }) => {
9       return true;
10     },
11   },
12   // Database schema + UI fields
13   fields: [
14     {
15       name: 'mainBooking',
16       type: 'relationship',
17       relationTo: 'bookings',
18       required: false,
19       // filterOptions: Dynamic filtering based on user
20       filterOptions: ({ user }) => ({
21         manager: { equals: user.id },
22       }),
23     },
24     {
25       name: 'relatedBookings',
26       type: 'join',
27       collection: 'bookings',
28       on: 'mainBooking',
29       where: {
30         status: {
31           not_in: ['cancelled'],
32         },
33       },
34     },
35     {
36       name: 'totalAmount',
37       type: 'number',
38       required: true,
39       admin: {
40         // Custom React component for the Admin UI
41         components: {
42           Field: CustomPriceEditor,
43         },
44         // Conditional display: only show if user has permission
45         condition: (data, siblingData, { user }) => {
46           return user.settings.bookings.canModifyPrices
47         },
48       },
49     },
50   ],
51 };;
```

Στο παραπάνω παράδειγμα:

- Το `relationship` field type δημιουργεί σχέσεις μεταξύ collections, υποστηρίζοντας one-to-one, one-to-many, και many-to-many relations.
- Το `join` field δημιουργεί «virtual» πεδία που υπολογίζονται δυναμικά χωρίς να αποθηκεύονται στη βάση, συνδέοντας εγγραφές μέσω της παράμετρου `on` και φιλτράροντας με `where`.
- Το `filterOptions` φιλτράρει δυναμικά τις επιλογές ενός `relationship` field βάσει του χρήστη ή άλλων παραμέτρων.
- Το `admin.components.Field` επιτρέπει την αντικατάσταση του default input με ένα custom React component (`CustomPriceEditor`).
- Το `admin.condition` καθορίζει πότε ένα πεδίο είναι ορατό στο UI βάσει δυναμικής λογικής (π.χ. δικαιώματα χρήστη).

Lifecycle Hooks και Direct Database Access

Το PayloadCMS παρέχει ένα ισχυρό σύστημα hooks που επιτρέπει την εκτέλεση κώδικα σε συγκεκριμένα σημεία του κύκλου ζωής ενός document:

- `beforeValidate`, `beforeChange`, `beforeRead`, `beforeDelete`
- `afterChange`, `afterRead`, `afterDelete`

Ωστόσο, για operations μεγάλης κλίμακας (mass imports), τα hooks μπορούν να δημιουργήσουν σημαντικό overhead. Για αυτό το λόγο, το PayloadCMS επιτρέπει **απευθείας πρόσβαση στη βάση δεδομένων**:

Listing 2.2: Standard Lifecycle vs Direct Database Access

```
1 // STANDARD: Full lifecycle - hooks execute, validation runs
2 const booking = await payload.create({
3   collection: 'bookings',
4   data: bookingData,
5 });
6
7 // DIRECT DB: Bypass lifecycle - for mass operations
8 // Uses Drizzle ORM under the hood
9 await payload.db.create({
10   collection: 'bookings',
11   data: dtoPayloadToPayloadDb(bookingData),
12 });
13
14 // Even lower level: Direct Drizzle access
```

```
15 const db = payload.db.drizzle;  
16 await db.insert(bookingsTable).values(bookingsArray);
```

Αυτή η ευελιξία είναι κρίσιμη για την απόδοση. Κατά την αρχική εισαγωγή ενός Manager (με χιλιάδες rates και bookings), η χρήση του `payload.db` αντί του `payload.create()` μπορεί να μειώσει τον χρόνο εκτέλεσης από ώρες σε λεπτά.

Drizzle ORM: Type-Safe Database Access

Το **Drizzle ORM** [5] είναι ένα σύγχρονο, ελαφρύ TypeScript ORM που χρησιμοποιείται από το PayloadCMS v3 για την επικοινωνία με τη βάση δεδομένων. Τα χαρακτηριστικά του περιλαμβάνουν:

- Αυτόματη παραγωγή TypeScript types από το schema
- Δυνατότητα raw SQL queries όταν χρειάζεται
- Υποστήριξη migrations για εξέλιξη του schema
- Υψηλή απόδοση συγκριτικά με βαρύτερα ORMs (όπως το Prisma)

2.1.5 PostgreSQL: Σχεσιακή Βάση Δεδομένων

Η PostgreSQL [6] είναι μια ισχυρή, open-source σχεσιακή βάση δεδομένων με δεκαετίες ανάπτυξης. Υποστηρίζεται από το PayloadCMS v3 ως η κύρια βάση δεδομένων (αντικαθιστώντας τη MongoDB που υποστηριζόταν στην έκδοση).

ACID Transactions: Η PostgreSQL εγγυάται Atomicity, Consistency, Isolation, Durability – κρίσιμο για εφαρμογές που απαιτούν συνέπεια δεδομένων κατά τον συγχρονισμό με εξωτερικά συστήματα.

Complex Queries: Η σχεσιακή δομή επιτρέπει πολύπλοκες αναζητήσεις (JOINS, aggregations) που είναι πιο αποδοτικές από τα αντίστοιχα document-based queries.

Relational Integrity: Υποστηρίζει foreign keys και referential integrity για αυστηρές σχέσεις μεταξύ οντοτήτων.

2.1.6 Shadcn UI, React και Zustand

Το Shadcn UI [7] είναι μια συλλογή επαναχρησιμοποιήσιμων UI components που διαφέρει από τις παραδοσιακές βιβλιοθήκες (όπως το Material UI). Αντί να εγκαθίσταται ως npm package, ο κώδικας αντιγράφεται απευθείας μέσα στο project, δίνοντας απόλυτη ελευθερία παραμετροποίησης.

React: Η δημοφιλέστερη JavaScript βιβλιοθήκη για τη δημιουργία επαναχρησιμοποιήσιμων UI components [8], που αποτελεί τη βάση του σύγχρονου frontend development.

Zustand: Ένα ελαφρύ state management library [9] που προσφέρει απλούστερη εναλλακτική στο Redux για τη διαχείριση της κατάστασης React εφαρμογών.

2.1.7 Swagger/OpenAPI και Code Generation

Το OpenAPI (πρώην Swagger) [10] είναι ένα πρότυπο για την περιγραφή RESTful APIs. Επιτρέπει την αυτόματη παραγωγή documentation και client SDKs.

Documentation Generation: Βιβλιοθήκες όπως το `next-swagger-doc` παράγουν αυτόματα API documentation από τον κώδικα.

Client Generation: Εργαλεία όπως το `openapi-typescript-codegen` παράγουν πλήρως τυποποιημένους TypeScript clients (SDKs) από OpenAPI specifications, εξασφαλίζοντας συγχρονισμό μεταξύ API και consumers.

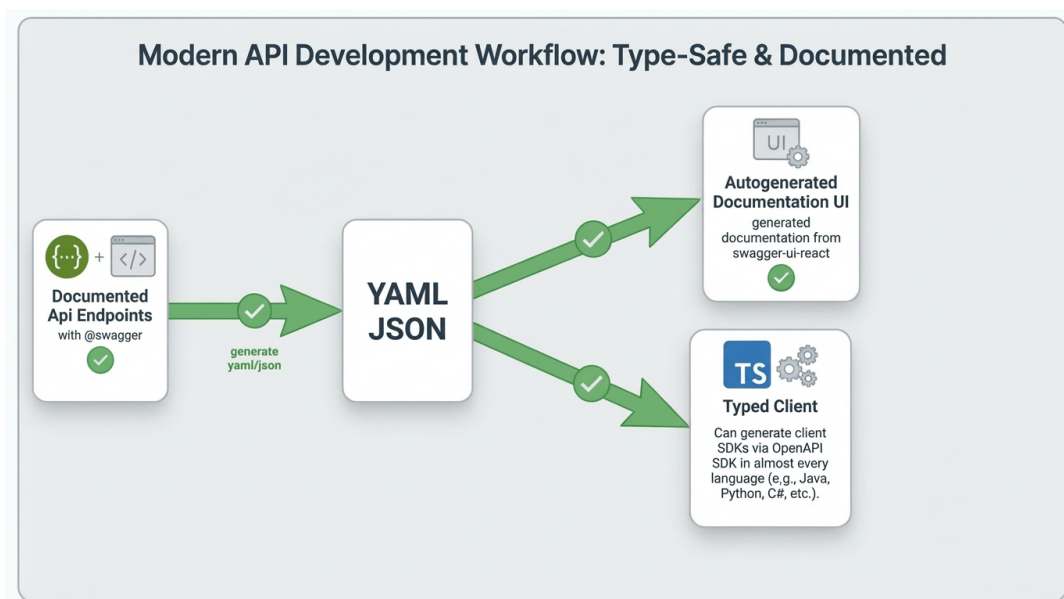


Figure 2.3: Ροή παραγωγής Client από OpenAPI/Swagger Documentation.

2.2 Large Language Models (LLMs)

Τα Large Language Models (LLMs) είναι νευρωνικά δίκτυα εκπαιδευμένα σε τεράστιες ποσότητες κειμένου [11]. Μέσω APIs από παρόχους όπως OpenAI, Anthropic, ή Google, επιτρέπουν την ενσωμάτωση δυνατοτήτων τεχνητής νοημοσύνης σε εφαρμογές. Στο πλαίσιο Property Management Systems (PMS), τα LLMs μπορούν να αναλύσουν

το context μιας κράτησης και να προτείνουν απαντήσεις, να μεταφράσουν μηνύματα σε πολλαπλές γλώσσες, ή να λειτουργήσουν ως virtual concierges σε ιστοσελίδες καταλυμάτων.

2.2.1 Vercel AI SDK και Streaming

Το **Vercel AI SDK** [12] είναι μια open-source βιβλιοθήκη που απλοποιεί την ενσωμάτωση LLMs σε web εφαρμογές. Η κύρια δυνατότητά του είναι το **Streaming** (ροή δεδομένων) – η απάντηση του AI εμφανίζεται σταδιακά (λέξη-λέξη) καθώς παράγεται, αντί να περιμένει ο χρήστης ολόκληρη την απάντηση. Αυτό βελτιώνει σημαντικά την εμπειρία χρήστη σε AI-powered chat interfaces.

2.2.2 OpenRouter: Ενοποιημένη Πρόσβαση σε LLM Μοντέλα

Το **OpenRouter** [13] είναι ένας ενδιάμεσος κόμβος (gateway) που παρέχει πρόσβαση σε πληθώρα LLM μοντέλων (GPT-4, Claude 3.5 Sonnet, Llama 3, κ.α.) μέσω ενός ενιαίου API. Αντί να συνδέεται μια εφαρμογή απευθείας με κάθε πάροχο (OpenAI, Anthropic, Meta), το OpenRouter επιτρέπει την αλλαγή μοντέλου χωρίς τροποποίηση κώδικα. Αυτό δίνει ευελιξία επιλογής μοντέλου βάσει αναγκών και budget.

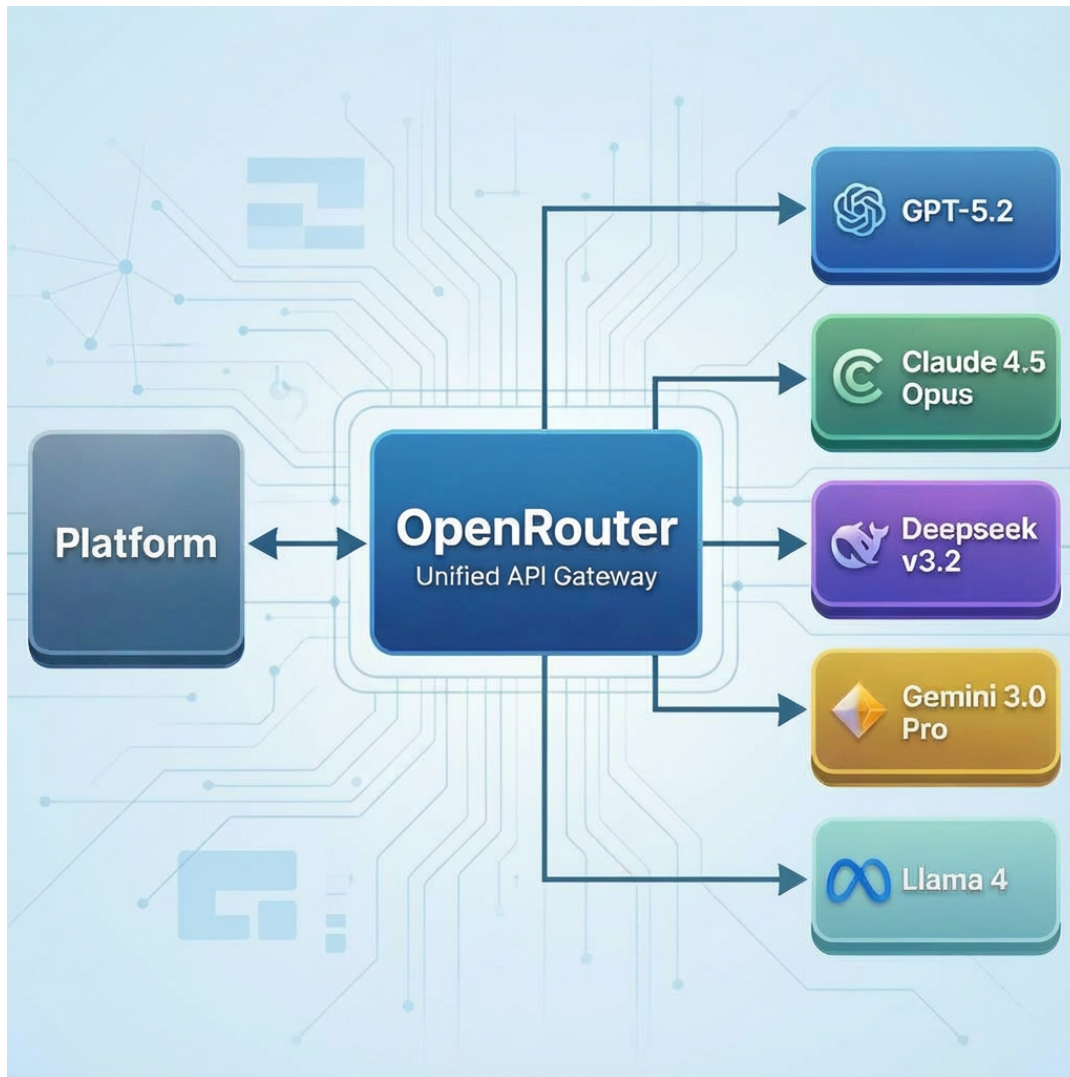


Figure 2.4: Αρχιτεκτονική OpenRouter: Ένας ενδιάμεσος κόμβος για πρόσβαση σε πληθώρα LLM μοντέλων.

2.3 Channel Managers και Διασυνδεσιμότητα

2.3.1 Τι είναι ο Channel Manager

Ο **Channel Manager** [14] είναι ένα λογισμικό που λειτουργεί ως ενδιάμεσος κόμβος μεταξύ των καταλυμάτων και των Online Travel Agencies (OTAs) όπως το Airbnb, το Booking.com, η Expedia, και η Vrbo. Ο κύριος ρόλος του είναι να διατηρεί **συνγχρονισμένες τις τιμές και τη διαθεσιμότητα** σε όλα τα κανάλια πώλησης σε πραγματικό χρόνο.

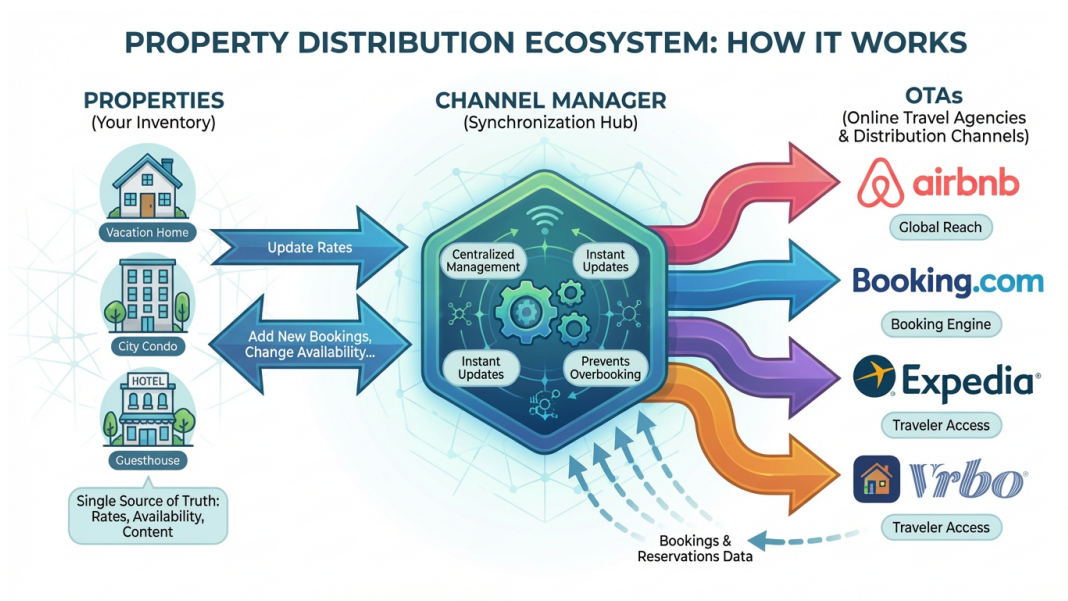


Figure 2.5: Αρχιτεκτονική Channel Manager ως ενδιάμεσου κόμβου.

Βασικές Λειτουργίες Channel Manager:

- **Συγχρονισμός Διαθεσιμότητας:** Όταν ένα δωμάτιο κλείνει στο Airbnb, ο Channel Manager ενημερώνει αυτόματα το Booking.com και όλα τα άλλα συνδεδεμένα κανάλια ότι το δωμάτιο δεν είναι πλέον διαθέσιμο. Αυτό αποτρέπει τις overbookings.
- **Συγχρονισμός Τιμών:** Επιτρέπει τον ορισμό τιμών από ένα κεντρικό σημείο, οι οποίες διανέμονται σε όλα τα κανάλια.
- **Κεντρική Διαχείριση Κρατήσεων:** Όλες οι κρατήσεις από διαφορετικά κανάλια συγκεντρώνονται σε ένα σημείο.
- **Επικοινωνία με Guests:** Πολλοί Channel Managers προσφέρουν unified in-box για μηνύματα από όλα τα κανάλια.

2.3.2 Γιατί είναι Απαραίτητοι οι Channel Managers

Ένα κρίσιμο ερώτημα είναι: *Γιατί δεν συνδέονται τα PMS απευθείας με τα OTAs;* Η απάντηση είναι πολύπλευρη:

1. **Περιορισμένη Πρόσβαση σε APIs:** Τα μεγάλα OTAs όπως το Airbnb και το Booking.com δεν παρέχουν δημόσια API access σε τρίτους developers. Η πρόσβαση στα APIs τους απαιτεί:
 - Εγκεκριμένη συνεργασία (Partnership Agreement)
 - Αυστηρές διαδικασίες πιστοποίησης

- Ελάχιστο volume κρατήσεων (συνήθως χιλιάδες ανά μήνα)
- Νομική οντότητα και ασφάλιση

Αυτό καθιστά την απευθείας σύνδεση αδύνατη για τις περισσότερες εταιρείες και startups.

2. **Τεχνική Πολυπλοκότητα:** Κάθε OTA έχει διαφορετικό API format, authentication mechanism, και business logic. Η υποστήριξη πολλαπλών OTAs απευθείας θα απαιτούσε τεράστια προσπάθεια ανάπτυξης και συντήρησης.
3. **Reliability και Support:** Οι Channel Managers έχουν dedicated teams για τη διατήρηση της σύνδεσης με τα OTAs, την αντιμετώπιση API changes, και την επίλυση προβλημάτων.

Παράδειγμα Πρακτικής Εφαρμογής: Ένα τυπικό PMS δεν μπορεί να λάβει απευθείας API access από το Airbnb ή το Booking.com. Ωστόσο, μέσω της σύνδεσης με έναν Channel Manager (όπως το Beds24), αποκτά πρόσβαση σε όλες τις κρατήσεις και τα μηνύματα από αυτά τα κανάλια. Ο Channel Manager λειτουργεί ως «γέφυρα» που μεταφράζει τις κλήσεις του PMS σε κλήσεις προς τα OTAs.

2.3.3 Beds24: Ένας Δημοφιλής Channel Manager

Το **Beds24** [15] είναι ένας από τους πιο διαδεδομένους Channel Managers στην αγορά, με ιδιαίτερη δημοτικότητα στην Ευρώπη. Τα κύρια χαρακτηριστικά του περιλαμβάνουν:

1. **Modern API:** Διαθέτει σύγχρονο, comprehensive API με υποστήριξη OpenAPI specification, επιτρέποντας την αυτόματη παραγωγή κώδικα (μέσω εργαλείων όπως το `swagger-typescript-api`).
2. **Webhook Support:** Υποστηρίζει real-time webhooks για νέες κρατήσεις και μηνύματα, επιτρέποντας instant ενημερώσεις αντί για polling.
3. **Πλήρες Feature Set:** Υποστηρίζει τιμές ανά ημέρα, restrictions (min/max stay), μηνύματα guests, και σύνδεση με πολλαπλά κανάλια (Airbnb, Booking.com, κ.α.).
4. **Ανταγωνιστική Τιμολόγηση:** Προσφέρει οικονομικές επιλογές συγκριτικά με άλλους Channel Managers της αγοράς.

Chapter 3

Μεθοδολογία και Ανάλυση Απαιτήσεων

3.1 Ρόλοι και Ιεραρχία Χρηστών

Το σύστημα σχεδιάστηκε με γνώμονα την ιεραρχία Manager-Owner:

1. **Platform Users (Admins):** Οι διαχειριστές της πλατφόρμας Lunarety. Έχουν καθολική εποπτεία.
2. **Managers:** Ο κεντρικός κόμβος. Συνδέουν το Channel Manager τους, δημιουργούν πλάνα χρέωσης και εισάγουν τους Owners τους. Η πρόσβαση τους καθορίζεται αυστηρά από τους Platform Users.
3. **Owners:** Πελάτες του Manager. Βλέπουν μόνο τα δικά τους ακίνητα και οικονομικά στοιχεία. Η πρόσβασή τους καθορίζεται αυστηρά από τον Manager.
4. **Staff:** Υπάλληλοι (καθαριστές, υποδοχή) που ανήκουν σε Manager ή προαιρετικά και σε Owner, με περιορισμένα δικαιώματα. Η πρόσβασή τους καθορίζεται αυστηρά από τον Manager ή Owner τους.

Το σύστημα υλοποιεί ένα εύκαμπτο μοντέλο δικαιωμάτων που επιτρέπει τον έλεγχο σε κάθε επίπεδο.

3.1.1 Τύποι Χρηστών (User Types)

Η πλατφόρμα υποστηρίζει τέσσερις διακριτούς τύπους χρηστών, όπως παρουσιάζονται στον Πίνακα 3.1.

Table 3.1: Τύποι χρηστών της πλατφόρμας

User Type	Περιγραφή	Parent	Children
platform-user	Administrators πλατφόρμας	Κανένας	Κανένας
manager	Property Managers	Platform	Owners, Staff
owner	Ιδιοκτήτες ακινήτων	Manager	Staff
stuff	Προσωπικό	Owner/Manager	Κανένας

Platform Users (Admins):

- Πλήρης πρόσβαση σε όλα τα δεδομένα της πλατφόρμας (Admin View)
- Δημιουργία και διαχείριση Platform Plans (πλάνα χρέωσης πλατφόρμας)
- Εποπτεία όλων των Users, Properties, χωρίς να είναι «γονείς» τους ιεραρχικά, αλλά έχοντας δυνατότητα διαχείρισης.

Managers:

- Σύνδεση με Channel Manager (Beds24)
- Εισαγωγή και διαχείριση Properties
- Δημιουργία Owners και Staff *
- Ορισμός δικαιωμάτων για τους υφισταμένους *
- Δημιουργία και διαχείριση Auto Actions *
- Δημιουργία Websites για τα ακίνητα *

Owners:

- Προβολή/επεξεργασία μόνο των ακινήτων που τους έχουν ανατεθεί *
- Δικαιώματα καθορίζονται πλήρως από τον Manager, Platform User
- Δημιουργία Staff που ανήκουν σε αυτούς *

Staff:

- Δυνατότητα αντίστοιχων δικαιωμάτων με τον Owner
- Συνήθως περιορισμένα δικαιώματα που οριστούν από τους Προϊστάμενους τους
- Χρήσιμοι για εργασίες καθαρισμού, check-in, κ.λπ.

* Οι δυνατότητες με αστερίσκο είναι διαθέσιμες μόνο εφόσον έχουν ανατεθεί τα αντίστοιχα δικαιώματα από τους Προϊσταμένους τους.

3.1.2 Ιεραρχικές Σχέσεις και Billing

Κάθε Owner ανήκει σε έναν Manager, ενώ το Staff ανήκει σε Manager και σε Owner. Η σχέση αυτή καθορίζει την πρόσβαση και τη ροή των χρεώσεων. Κάθε Manager και Owner συνδέεται με ένα Plan που καθορίζει τις χρεώσεις και τα features.

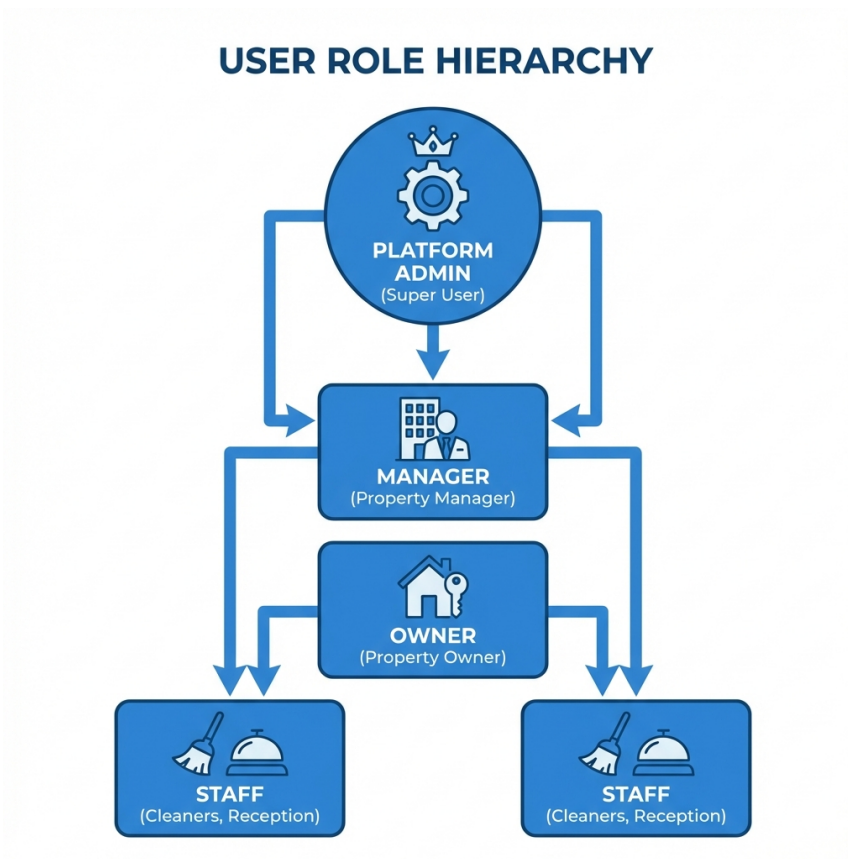


Figure 3.1: Ιεραρχική δομή ρόλων χρηστών στην πλατφόρμα.

3.2 Αρχιτεκτονική Συστήματος (Monorepo)

Η πλατφόρμα υλοποιείται ως Monorepo, το οποίο περιλαμβάνει:

- Τον πυρήνα της εφαρμογής (PayloadCMS + Next.js).
- Τους Webhook Handlers για την επικοινωνία με τρίτα συστήματα.
- Τα Cron Jobs για τις χρονοπρογραμματισμένες εργασίες.

Η πλατφόρμα εξασφαλίζει συνοχή μεταξύ των διαφορετικών components και διευκολύνει την κοινή χρήση τύπων και utilities.

3.2.1 Δομή Monorepo

Η πλατφόρμα αποτελείται από δύο ξεχωριστά repositories:

Platform Repository (lunarety)

Το κύριο repository περιέχει τον πυρήνα της πλατφόρμας:

Listing 3.1: Δομή καταλόγων της Πλατφόρμας (lunarety)

```

1 lunarety-v2/
2 |-- src/
3 |   |-- app/                # Next.js App Router
4 |   |   |-- (frontend)/     # Frontend routes
5 |   |   +-- (payload)/      # PayloadCMS admin routes
6 |   |-- apis/               # External API integrations
7 |   |   +-- channel-managers/ # Beds24 API, DTOs, sync
8 |   |-- collections/        # PayloadCMS collections
9 |   |-- crons/              # Scheduled jobs
10 |  |-- endpoints/          # Custom API endpoints
11 |  |-- lib/                 # Shared utilities
12 |  +-- payload.config.ts    # PayloadCMS configuration
13 +-- package.json

```

Website Template Repository (lunarety-website)

Το δεύτερο repository είναι δημόσια διαθέσιμο στο GitHub¹ και λειτουργεί ως template για τις ιστοσελίδες των καταλυμάτων:

Listing 3.2: Δομή καταλόγων του Website Template (lunarety-website)

```

1 lunarety-website/
2 |-- src/
3 |   |-- app/                # Next.js App Router
4 |   |   |-- page.tsx        # Home page (property search)
5 |   |   |-- properties/     # Property listing & details
6 |   |   +-- booking/        # Booking management pages
7 |   |-- components/         # React UI components
8 |   |   |-- ui/             # Shadcn UI components
9 |   |   +-- chat/           # AI Chat Bubble component
10 |  +-- lib/                 # Utilities & API clients
11 |-- public/                 # Static assets
12 +-- package.json

```

¹<https://github.com/barkarian/lunarety-website>

3.2.2 Βασικά Components

PayloadCMS Core:

- Headless CMS με PostgreSQL backend
- Αυτόματη δημιουργία REST και GraphQL API
- Admin panel για διαχείριση δεδομένων
- Type-safe collection definitions

Next.js App Router:

- Server-side rendering για SEO
- Server Actions για data mutations
- Middleware για authentication
- Dynamic routing για Calendar, Messages, Settings

Channel Manager API Layer:

- `ChannelManagerApi` object – central export point
- DTOs για μετατροπή Beds24 ↔ Payload formats
- Abstraction layer για μελλοντική υποστήριξη άλλων Channel Managers

Το κρίσιμο σημείο αυτής της αρχιτεκτονικής είναι ότι ολόκληρη η πλατφόρμα χρησιμοποιεί αυτές τις συναρτήσεις **χωρίς να είναι εξαρτημένη από το Beds24**. Οι υπόλοιπες μονάδες της εφαρμογής (hooks, cron jobs, webhooks) καλούν τα generic methods του `ChannelManagerApi` object, τα οποία εσωτερικά μεταφράζονται στις αντίστοιχες κλήσεις του συγκεκριμένου Channel Manager. Αυτός ο σχεδιασμός ακολουθεί το πρότυπο **Adapter Pattern** [16, 17].

3.2.3 External Integrations

Table 3.2: External Services και Integrations

Service	Σκοπός	Module
Beds24 API	Channel Manager	apis/channel-managers/beds24/
Vercel AI SDK	AI text improvements	lib/ai/
OpenRouter	LLM API proxy	lib/ai/
Telegram Bot API	User notifications	lib/telegram/
Resend	Email sending	lib/email/

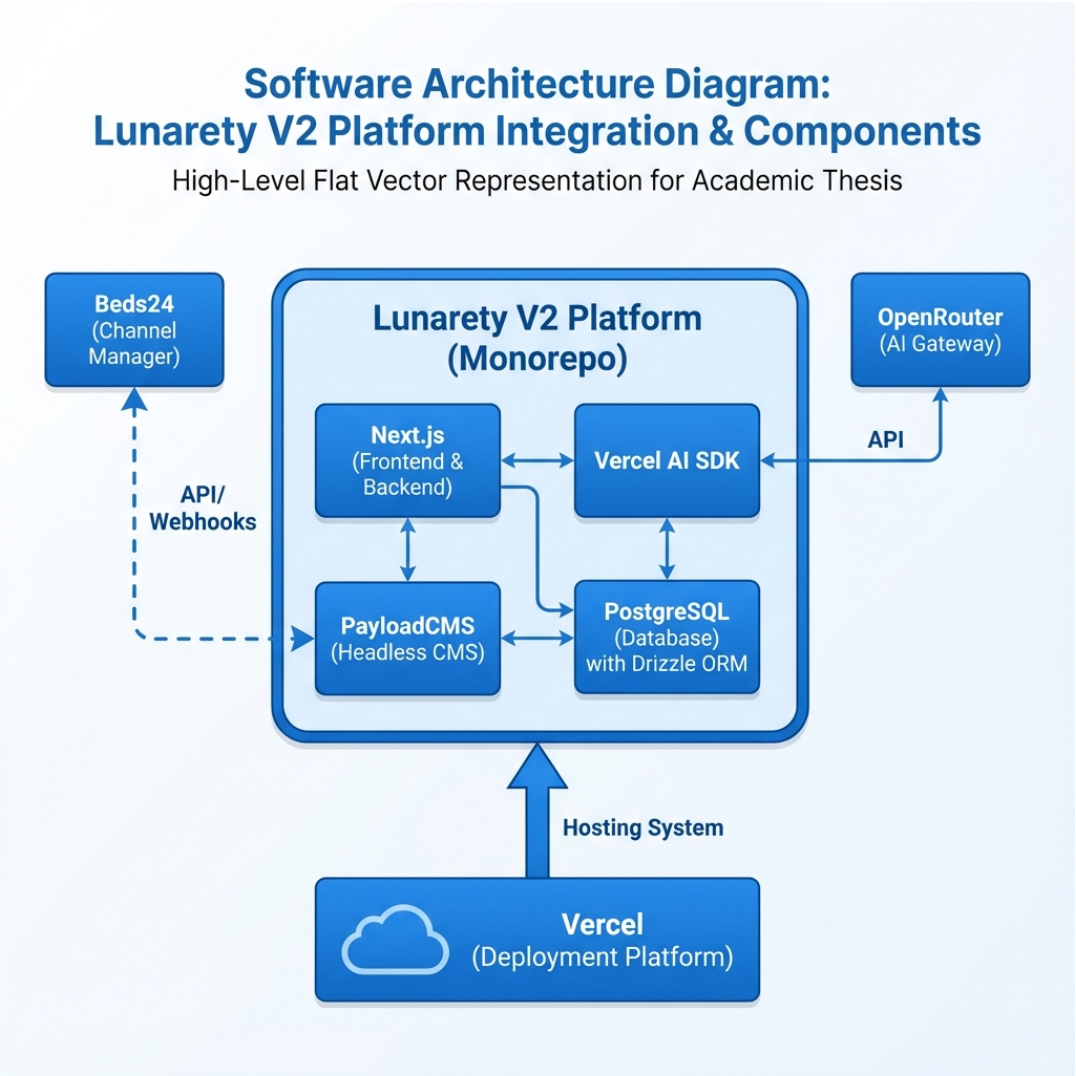


Figure 3.2: Διάγραμμα αρχιτεκτονικής συστήματος (Monorepo).

Παράλληλα, το `lunarety-v2-website` αποτελεί ένα ξεχωριστό public project στο GitHub, σχεδιασμένο να λειτουργεί ως template για τις ιστοσελίδες των καταλυμάτων,

το οποίο όμως συνδέεται άμεσα με την πλατφόρμα μέσω API.

3.3 Μοντέλο Δεδομένων και Σχέσεις

Η βάση δεδομένων (PostgreSQL) οργανώνεται γύρω από τις εξής κύριες οντότητες:

- **Users:** Περιλαμβάνει ιεραρχικές σχέσεις (parent-child) για τη σύνδεση Manager-Owner-Staff.
- **Properties:** Τα ακίνητα, τα οποία συνδέονται με `propId` του Beds24 και ανήκουν σε έναν Manager και (προαιρετικά) σε έναν Owner.
- **Bookings:** Κεντρικός πίνακας κρατήσεων με πλήρη στοιχεία πελάτη και οικονομικά δεδομένα.
- **Auto-Actions:** Πίνακας κανόνων αυτοματισμού.

3.3.1 Κύριες Οντότητες (Collections)

Η βάση δεδομένων (PostgreSQL) οργανώνεται γύρω από τις κύριες collections που παρουσιάζονται στον Πίνακα 3.3.

Table 3.3: Κύριες Collections της βάσης δεδομένων

Collection	Περιγραφή	Σημαντικά Fields
users	Χρήστες πλατφόρμας	type, managedBy, ownedBy, settings
property	Καταλύματα	manager, owner, channelPropertyId, rooms
bookings	Κρατήσεις	property, resource, pricing, messages
property-rate	Τιμές ανά ημέρα	property, date, channelRoomId, availability
billing	Χρεώσεις/Τιμολόγια	type, ownerDebtor, managerCreditor
auto-actions	Κανόνες αυτοματισμού	type, triggerType, template
plans	Πλάνα χρέωσης	planCommission, features
website	Generated websites	apiKey, type, properties

3.3.2 Σχέσεις Μεταξύ Οντοτήτων

Property ↔ Users:

- Κάθε Property έχει έναν `manager` (υποχρεωτικό)
- Κάθε Property μπορεί να έχει έναν `owner` (προαιρετικό)

Bookings ↔ Property ↔ Users:

- Κάθε Booking ανήκει σε ένα Property
- Το Access control βασίζεται στη σχέση Property → Manager/Owner

Property Rates ↔ Property ↔ Bookings:

- Κάθε Rate συνδέεται με μια ημέρα και ένα δωμάτιο
- Τα Bookings συνδέονται με τα rates μέσω `relatedRates`
- Join field `bookingsStayingOnThisNight` για reverse lookup

Auto Actions ↔ Booking ↔ Users/Properties:

- Κάθε Auto Action ενεργοποιείται (Trigger) από γεγονότα που αφορούν ένα **Booking**
- Συνδέει και χρησιμοποιεί δεδομένα από πολλά Properties και Users
- Τα Properties μπορούν να «εγγραφούν» (subscribe) σε Auto Actions

3.3.3 Hook System και Lifecycle

Το PayloadCMS παρέχει ένα ισχυρό σύστημα hooks που χρησιμοποιείται εκτενώς (Πίνακας 3.4).

Table 3.4: Hook Types και χρήση στην πλατφόρμα

Hook Type	Εκτελείται	Χρήση
<code>beforeChange</code>	Πριν την αποθήκευση	Sync με CM, Commission calc
<code>afterChange</code>	Μετά την αποθήκευση	Touch rates, Trigger actions
<code>beforeRead</code>	Πριν την ανάγνωση	Data transformation
<code>afterRead</code>	Μετά την ανάγνωση	Computed fields, formatting

Παράδειγμα Hook Chain για Booking Creation:

1. `linkRatesToBooking (beforeChange)` – Σύνδεση με property rates
2. `syncChannelBooking_beforeChangeHook (beforeChange)` – Υπολογισμός commissions, sync με Beds24
3. `touchNightlyRates (afterChange)` – Ενημέρωση σχετικών rates

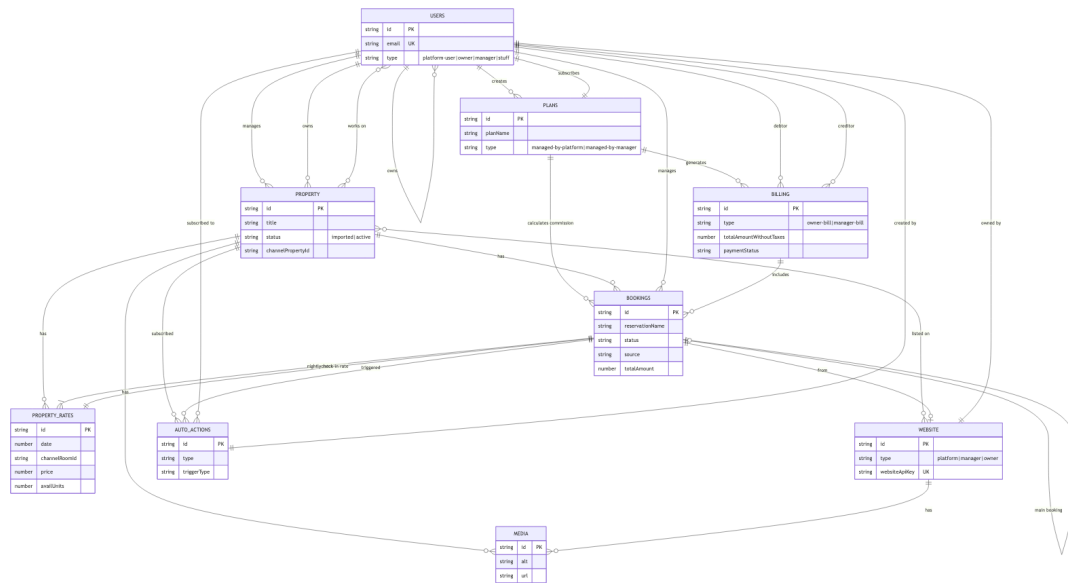


Figure 3.3: Entity Relationship Diagram (ERD) του μοντέλου δεδομένων της πλατφόρμας.

Chapter 4

Υλοποίηση

Πριν αναλύσουμε τις επιμέρους λειτουργίες της πλατφόρμας, είναι σημαντικό να κατανοήσουμε τη θεμελιώδη αρχιτεκτονική που διέπει τον τρόπο με τον οποίο τα δεδομένα αλληλεπιδρούν μεταξύ τους. Το σύστημα lifecycle hooks αποτελεί τον πυρήνα αυτής της αρχιτεκτονικής και καθορίζει πώς οι αλλαγές σε ένα collection επηρεάζουν τα υπόλοιπα.

4.1 Αρχιτεκτονική Lifecycle Hooks Μεταξύ Collections

Ένα από τα πιο κρίσιμα αρχιτεκτονικά στοιχεία της πλατφόρμας είναι ο τρόπος με τον οποίο οι διαφορετικές συλλογές (Collections) αλληλεπιδρούν μεταξύ τους μέσω του συστήματος lifecycle hooks. Το Σχήμα 4.1 απεικονίζει αναλυτικά τις ροές που ενεργοποιούνται κατά την αλλαγή δεδομένων σε κάθε collection: **Users**, **Properties**, **Bookings** και **Billings**.



Figure 4.1: Διάγραμμα ροής lifecycle hooks μεταξύ των κύριων Collections της πλατφόρμας. Κάθε collection ενεργοποιεί συγκεκριμένες ροές βάσει συνθηκών, με τελικό σημείο σύγκλισης τα PropertyRates.

Όπως φαίνεται στο διάγραμμα, κάθε collection έχει το δικό του `on:change` trigger που ενεργοποιεί διαφορετικές διαδρομές ανάλογα με τις συνθήκες. Τα βασικά αρχιτεκτονικά patterns που προκύπτουν από αυτή τη σχεδίαση είναι τα εξής:

4.1.1 Βασικά Αρχιτεκτονικά Patterns

Στρατηγική «Κύματος» (Ripple Effect): Η αρχιτεκτονική είναι σχεδιασμένη ώστε μια αλλαγή στο ανώτερο επίπεδο να διαδίδεται προς τα κάτω στη μικρότερη μονάδα (το Rate). Τόσο οι αλλαγές σε Properties όσο και σε Bookings καταλήγουν να ενεργοποιούν το lifecycle των `propertyRates`, διασφαλίζοντας ότι η λογική τιμολόγησης είναι πάντα ακριβής.

Αποτροπή Βρόχων (`skipChannelSync`): Ο αμφίδρομος συγχρονισμός (Local DB ↔ Beds24) είναι επιρρεπής σε «ηχώ». Το `skipChannelSync` flag αποτρέπει το σύστημα από το να προσπαθήσει να ενημερώσει πίσω τον Channel Manager με δεδομένα που μόλις έλαβε από αυτόν.

Ισορροπία Απόδοσης vs Ακρίβειας: Σε πολλά ORMs, η εκτέλεση `updateMany` παρακάμπτει τα lifecycle hooks. Το σύστημα εκτελεί τις ενημερώσεις rates «ένα-ένα» για να διασφαλίσει ότι οι event listeners ενεργοποιούνται για κάθε record. Αν και αυτή η προσέγγιση είναι αργή, συνήθως εκτελείται μόνο μία φορά ανά Property κατά το onboarding.

Διαχείριση Κατάστασης (State Management): Η διάκριση μεταξύ `imported` και `active` properties επιτρέπει την ασφαλή εισαγωγή δεδομένων από τον Channel Manager (staging) πριν ξεκινήσει η επεξεργασία live κρατήσεων.

Με αυτή την αρχιτεκτονική ως βάση, οι επόμενες ενότητες αναλύουν λεπτομερώς τις επιμέρους υλοποιήσεις κάθε λειτουργίας.

4.2 Διαχείριση Κρατήσεων και Συγχρονισμός

Η διασύνδεση με το Beds24 είναι κρίσιμη και ακολουθεί μια συγκεκριμένη ροή:

1. **Σύνδεση Λογαριασμού:** Ο Manager εισάγει το API Key του Beds24.
- 2.
3. **Token Refresh:** Μια φορά την ημέρα και κατά το Initialization γίνεται αυτόματη ανανέωση του Beds24 token κάθε Manager μέσω του refresh token, ώστε να διασφαλίζεται απρόσκοπτη επικοινωνία με το Channel Manager.
4. **Initial Import:** Κατά την εισαγωγή του κλειδιού, το σύστημα εκτελεί μια μαζική εισαγωγή (Import) όλων των Properties και των Rates που είναι ορισμένα στο Channel Manager.
5. **Webhook Configuration:** Το σύστημα αυτόματα ενημερώνει τις ρυθμίσεις του Channel Manager (Beds24) ώστε να στέλνει Webhooks στο endpoint της πλατφόρμας (`/api/webhooks/channel-managers/beds24/bookings`).
6. **Async Updates:** Από εκείνη τη στιγμή, κάθε αλλαγή (νέα κράτηση, αλλαγή ημερομηνίας) έρχεται ασύγχρονα μέσω Webhook.

Η διαχείριση κρατήσεων αποτελεί τον πυρήνα της πλατφόρμας και απαιτεί προσεκτικό σχεδιασμό για να εξασφαλιστεί η ακεραιότητα των δεδομένων, η απόδοση και η συνέπεια μεταξύ της τοπικής βάσης δεδομένων και του Channel Manager.

4.2.1 Αρχική Σύνδεση και Ρύθμιση Webhooks

Η διασύνδεση με το Beds24 ακολουθεί μια συγκεκριμένη ροή αρχικοποίησης:

1. **Σύνδεση Λογαριασμού:** Ο Manager εισάγει το Refresh Token του Beds24 στις ρυθμίσεις του λογαριασμού του.
2. **Token Exchange:** Το σύστημα αυτόματα ανταλλάσσει το Refresh Token για ένα Access Token μέσω του `b24ApiRefreshToken()`.
3. **User ID Retrieval:** Ανακτάται το `channelManagerUserId` από το Beds24.
4. **Initial Property Import:** Εκτελείται μαζική εισαγωγή όλων των Properties και των Property Rates.
5. **Webhook Configuration:** Για κάθε Property, ρυθμίζεται αυτόματα το Beds24 να στέλνει Webhooks.

Listing 4.1: Αυτόματη ρύθμιση webhooks κατά την εισαγωγή properties

```
1 // Automatic webhook setup during property import
2 await api.properties.propertiesCreate(
3   propertiesWithRates.map(({ convertedProperty }) => ({
4     id: Number(convertedProperty.channelPropertyId),
5     webhooks: {
6       url:
7         `${process.env.NEXT_PUBLIC_URL}/api/webhooks/channel-managers/beds24/bookings`,
8       customHeader: `secretPropertyKey:
9         ${convertedProperty.secretPropertyKey}`,
10      version: "twoWithPersonalData",
11      additionalData: "none",
12    },
13  })),
14 );
```

4.2.2 Μέθοδοι Εισαγωγής Κρατήσεων

Η πλατφόρμα υποστηρίζει τέσσερις διαφορετικές μεθόδους δημιουργίας/ενημέρωσης κρατήσεων:

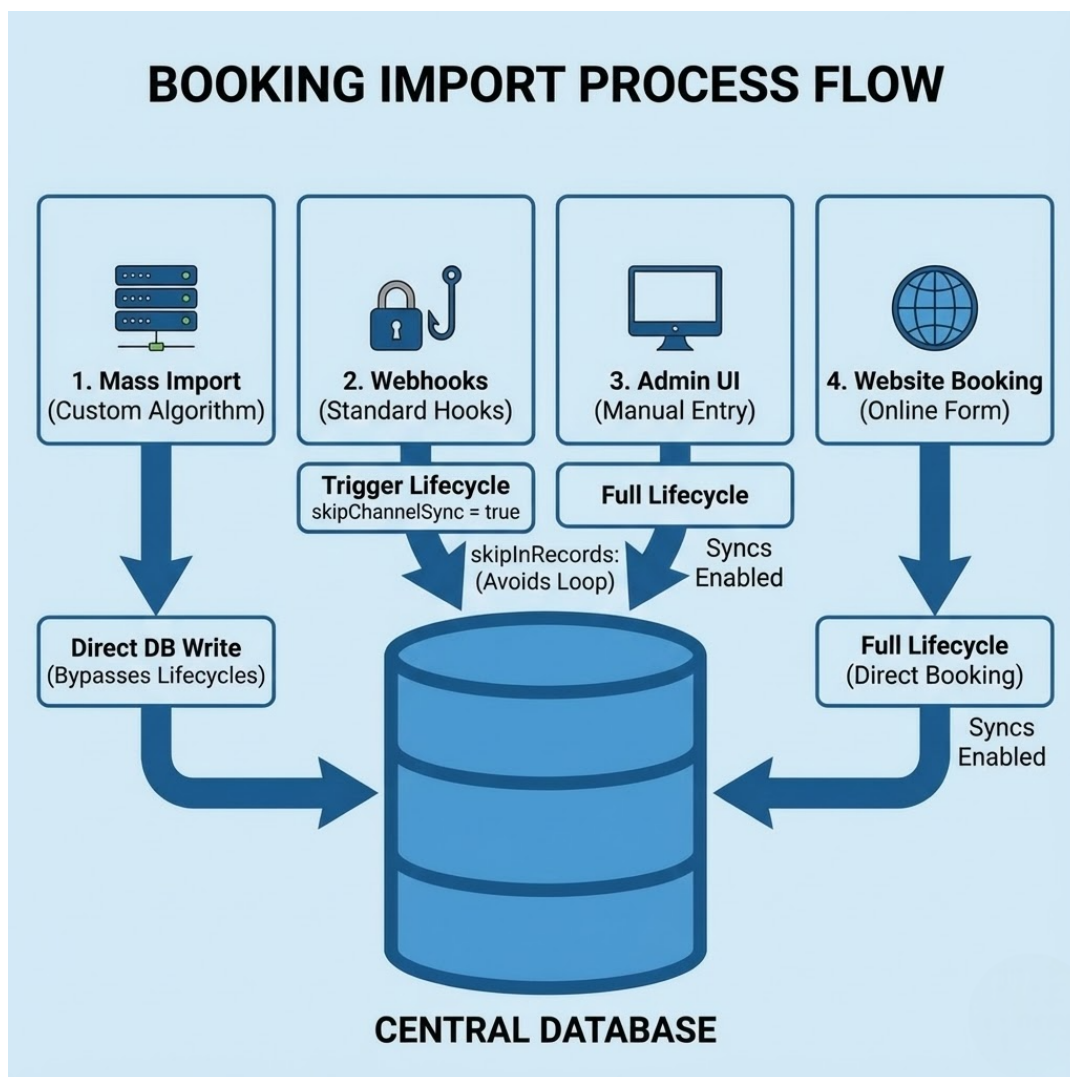


Figure 4.2: Μέθοδοι εισαγωγής κρατήσεων στην πλατφόρμα.

A) Mass Import (Property Syncing)

Η **μαζική εισαγωγή bookings** πραγματοποιείται όταν ένα Property μεταβαίνει από την κατάσταση `imported` στην `active` (ενεργοποίηση του καταλύματος). Κατά τη διαδικασία αυτή, η πλατφόρμα χρειάζεται να εισάγει εκατοντάδες ή χιλιάδες κρατήσεις από το Beds24. Χρησιμοποιείται η `payload.db.create()` και `payload.db.updateOne()` αντί των `standard methods`, παρακάμπτοντας τα `hooks` για βέλτιστη απόδοση.

Listing 4.2: Mass import με direct DB operations

```
1 const syncBookingsOperations = async (
2   bookingsToCreate: RequiredDataFromCollectionSlug<"bookings">[],
3   bookingsToUpdate: Array<{
4     id: number;
5     data: RequiredDataFromCollectionSlug<"bookings">;
6   }>,

```

```
7   bookingsToDelete: Booking[]
8 ) => {
9   // CREATE - Direct DB, bypassing lifecycle
10  for (const booking of bookingsToCreate) {
11    await payload.db.create({
12      collection: "bookings",
13      data: dtoPayloadToPayloadDb(booking),
14    });
15  }
16
17  // UPDATE - Direct DB, bypassing lifecycle
18  for (const { id, data } of bookingsToUpdate) {
19    await payload.db.updateOne({
20      collection: "bookings",
21      id: id,
22      data: dtoPayloadToPayloadDb(data),
23    });
24  }
25 };
```

Μετά την εισαγωγή των bookings, ενεργοποιείται η ενημέρωση **όλων των σχετικών PropertyRates** για να αντικατοπτριστεί η διαθεσιμότητα. Αν και αυτή η διαδικασία δεν είναι βέλτιστη (καθώς ενεργοποιεί mass updates σε χιλιάδες rate records), δεν έχει αναπτυχθεί ακόμα χαμηλότερου επιπέδου υλοποίηση – η βελτιστοποίηση αυτή αναλύεται στην Ενότητα [4.2.5](#).

Σημειώνεται ότι η μαζική εισαγωγή **εκτελείται μόνο μία φορά** κατά την ενεργοποίηση του property. Μετά την αρχική εισαγωγή, όλες οι ενημερώσεις (δημιουργία, τροποποίηση, ακύρωση κρατήσεων) λαμβάνονται μέσω **webhooks** σε πραγματικό χρόνο, οπότε η απόδοση είναι ικανοποιητική.

B) Webhook Updates (Channel Manager Triggers)

Όταν μια κράτηση δημιουργείται ή ενημερώνεται στο Beds24, η πλατφόρμα λαμβάνει ένα webhook notification. Κρίσιμο είναι το skipChannelSync flag:

Listing 4.3: Αποτροπή infinite loop με skipChannelSync flag

```
1  const createBookingRes = await payload.create({
2    collection: "bookings",
3    data: { ...dtoPayloadToPayloadDb(newBooking), skipChannelSync: true },
4    req: { transactionID: transactionId },
5  });
```

Γ) UI-Based Booking Creation

Όταν ένας χρήστης δημιουργεί κράτηση μέσω του UI, εκτελείται το πλήρες lifecycle συμπεριλαμβανομένου του υπολογισμού προμηθειών:

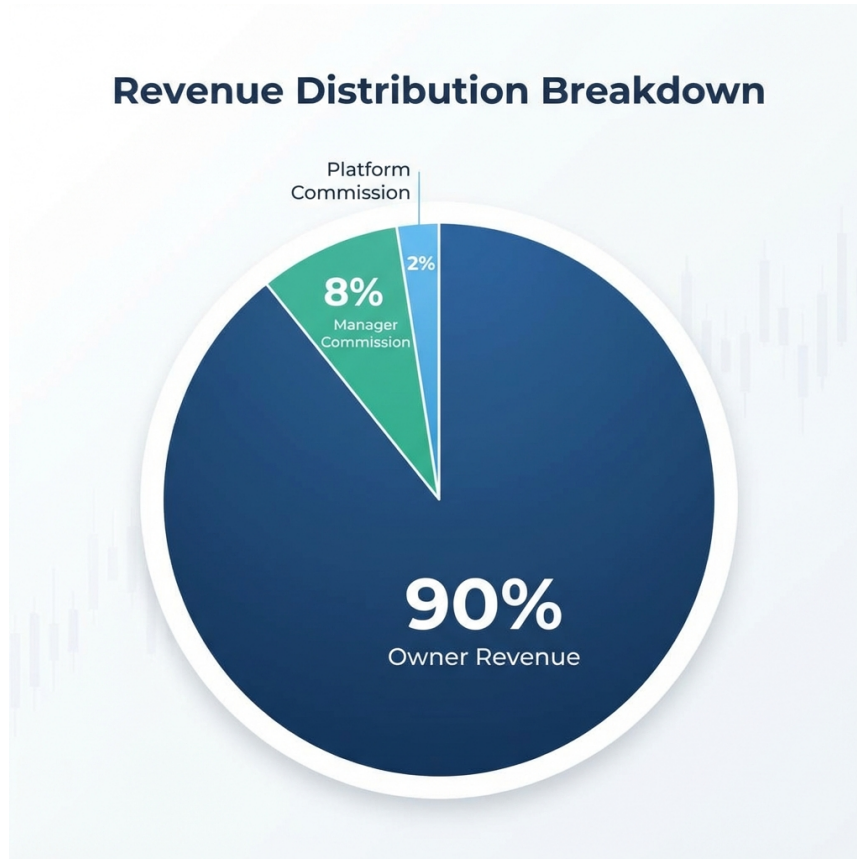


Figure 4.3: Τριπλή δομή προμηθειών (Three-Tier Commission Structure).

Listing 4.4: Υπολογισμός commission

```
1 export const computeCommissionOfPlan = (  
2   booking: Partial<Booking>,  
3   plan: Plan  
4 ): number => {  
5   const bookingTotal = booking.pricing?.totalAmount ?? 0;  
6  
7   if (booking.excludedFromCommission) return 0;  
8  
9   if (plan.planCommission?.commissionType === "percentage") {  
10    const percentage = plan.planCommission.commissionPercentage ?? 0;  
11    return Math.round(bookingTotal * (percentage / 100) * 100) / 100;  
12  } else {  
13    return plan.planCommission?.fixedCommission ?? 0;  
14  }
```

```
15 };
```

Δ) Website Booking Creation

Για κρατήσεις από το generated website, το πεδίο `source` ορίζεται αντίστοιχα (`websitePlatform`, `websiteManager`, `websiteOwner`).

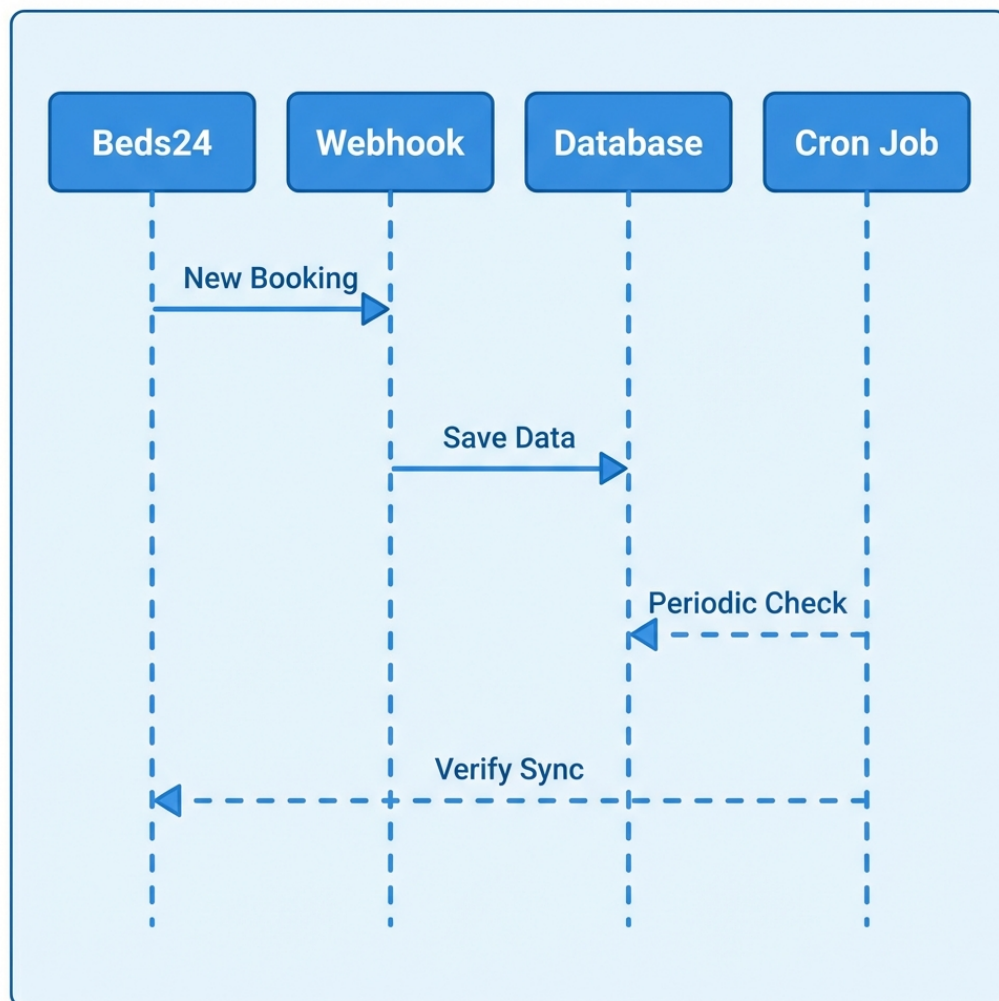


Figure 4.4: Ροή δεδομένων συγχρονισμού κρατήσεων με το Beds24.

4.2.3 Σύστημα Σύνδεσης με Property Rates

Κάθε κράτηση συνδέεται με τα αντίστοιχα `PropertyRate` records μέσω του `relatedRates` field. Αυτό είναι κρίσιμο για:

- Τον υπολογισμό διαθεσιμότητας (π.χ. ένα rate με 3 units και με 3 booked unit δεν είναι διαθέσιμο)

- Την ενημέρωση του ημερολογίου

linkRatesToBooking Hook: Για να διασφαλίσουμε ότι κάθε κράτηση συνδέεται με τα σωστά property rates, χρησιμοποιούμε το hook `linkRatesToBooking`. Αυτό το hook, κατά τη δημιουργία ή την επεξεργασία μιας κράτησης, εντοπίζει και συσχετίζει με την κράτηση όλα τα rate που αφορούν τις επιλεγμένες ημερομηνίες διαμονής (`checkIn`, `checkOut`).

Listing 4.5: linkRatesToBooking Hook

```
1 export const linkRatesToBooking: CollectionBeforeChangeHook<Booking> =  
  async ({  
2    data,  
3    req,  
4  }) => {  
5    // Εύρεση σχετικών rates για τη διάρκεια της κράτησης  
6    const rates = await payload.find({  
7      collection: "property-rates",  
8      where: {  
9        channelRoomId: { equals: data.resource.roomAndUnit.roomId },  
10       date: {  
11         greater_than_equal: data.resource.checkIn,  
12         less_than: data.resource.checkOut,  
13       },  
14     },  
15     limit: 10000,  
16   });  
17  
18   data.relatedRates = {  
19     nightlyRates: rates.docs.map((rate) => rate.id),  
20     checkInRate: rates.docs.find((rate) => rate.date ===  
21       data.resource.checkIn  
22     ).id,  
23   };  
24   return data;  
25 };
```

touchNightlyRates Hook (Cascade Update): Μετά την αποθήκευση μιας κράτησης, χρειάζεται να ενημερωθούν τα σχετικά rates ώστε να αντικατοπτρίζουν τη νέα κατάσταση. Αυτό γίνεται με ένα «touch» update:

Listing 4.6: touchNightlyRates Hook

```
1 export const touchNightlyRates: CollectionAfterChangeHook<Booking> =  
  async ({
```

```
2   doc,
3   req,
4 }) => {
5   const rateIds =
6     doc.relatedRates?.nightlyRates?.map((rate) =>
7       typeof rate === "object" ? rate.id : rate
8     ) || [];
9
10  // "Touch" update - ενεργοποιείται hooks του property-rates
11  await Promise.all(
12    rateIds.map((id) =>
13      payload.update({
14        collection: "property-rates",
15        id,
16        data: {}, // Κενό update, μόνο για να τρέξουν τα hooks
17        req: { transactionID: req.transactionID },
18      })
19    )
20  );
21  return doc;
22 };
```

4.2.4 Διαχείριση Transactions και Ασφάλεια Δεδομένων

Όλες οι κρίσιμες λειτουργίες εκτελούνται εντός database transactions για να εξασφαλιστεί η ακεραιότητα:

Listing 4.7: Χρήση Transactions

```
1 const transactionId = await payload.db.beginTransaction();
2 try {
3   // Πολλαπλές operations...
4   await payload.db.commitTransaction(transactionId);
5 } catch (error) {
6   await payload.db.rollbackTransaction(transactionId);
7   throw error;
8 }
```

4.2.5 Εισαγωγή Properties και Βελτιστοποίηση Property Rates

Η εισαγωγή καταλυμάτων και των τιμών τους αποτελεί μία από τις πιο απαιτητικές λειτουργίες σε επίπεδο απόδοσης. Για ένα τυπικό κατάλυμα με 4 δωμάτια, το σύστημα χρειάζεται να εισάγει περίπου 5.840 records (4 δωμάτια $\times$ 1.460 ημέρες = 4 χρόνια rates).

Μαζικές Εισαγωγές και Απευθείας Πρόσβαση στη Βάση: Γενικά, για τις μαζικές εισαγωγές (mass imports) – είτε πρόκειται για bookings είτε για properties και rates – χρησιμοποιούνται **custom algorithms που δεν βασίζονται καθόλου σε lifecycle hooks** και προσπελαίνουν απευθείας τη βάση δεδομένων. Αυτή η προσέγγιση είναι απαραίτητη για λόγους απόδοσης, καθώς η εκτέλεση του πλήρους lifecycle για χιλιάδες records θα ήταν απαγορευτικά αργή.

Αρχικοποίηση και Καθημερινή Συντήρηση: Η μαζική εισαγωγή properties και rates πραγματοποιείται κατά την **αρχική σύνδεση** (initialization) του Manager με τον Channel Manager. Επιπλέον, τα **access tokens ανανεώνονται καθημερινά** μέσω του everyNightCron, διασφαλίζοντας την αδιάλειπτη επικοινωνία με το Beds24 API.

Στρατηγική Εισαγωγής Rates: Το σύστημα εισάγει τιμές για ένα παράθυρο 4 ετών (2 χρόνια πίσω + 2 χρόνια μπροστά) για κάθε δωμάτιο. Το Beds24 στέλνει τα rates σε συμπυκνωμένη μορφή (date ranges), π.χ. «από 1/1 έως 31/1 με τιμή €100», τα οποία το σύστημα επεκτείνει σε μεμονωμένες ημέρες.

Ευρετηρίαση (Indexing) για Βελτιστοποίηση: Ο πίνακας **PropertyRates** έχει **ευρετηριαστεί (indexed)** στα πεδία που διαβάζονται και ενημερώνονται πιο συχνά, όπως το date και το channelRoomId. Αυτό είναι κρίσιμο διότι όταν ενημερώνεται ένα booking, το σύστημα χρειάζεται να ενημερώσει **όλα τα προηγούμενα και τρέχοντα related rates** – μια λειτουργία που εκτελείται πολύ συχνά.

Ενημέρωση Rates κατά την Εισαγωγή: Κατά την εισαγωγή ή τον συγχρονισμό, ενημερώνονται **όλα τα room rates του property**. Αυτή είναι μια βαριά λειτουργία (ενδεχομένως χιλιάδες records), και για αυτό χρησιμοποιείται ειδικός αλγόριθμος που προσπελαίνει απευθείας τη βάση δεδομένων χωρίς να ενεργοποιεί το lifecycle για κάθε record ξεχωριστά.

Αλγόριθμος Σύγκρισης Rates: Κατά τον συγχρονισμό, το σύστημα συγκρίνει τα νέα rates με τα υπάρχοντα στη βάση, δημιουργώντας maps για γρήγορη αναζήτηση και κατηγοριοποιώντας τα rates σε: ratesToCreate (νέα), ratesToUpdate (τροποποιημένα), και ratesToDelete (διαγραμμένα).

Βελτιστοποίηση Απόδοσης: Για την εισαγωγή χιλιάδων rates, χρησιμοποιούνται direct DB operations:

Table 4.1: Στρατηγικές Βελτιστοποίησης Rates

Μέθοδος	Χρήση	Πλεονεκτήματα
<code>payload.db.create()</code>	Νέα rates	Παρακάμπτει hooks, ~10x πιο γρήγορο
<code>payload.db.updateOne()</code>	Ενημέρωση rates	Αποφεύγει cascade updates
<code>payload.update()</code>	Μεμονωμένες αλλαγές	Πλήρες lifecycle, ενεργοποιεί hooks

Γιατί η Τρέχουσα Υλοποίηση είναι Αποδεκτή: Παρά τα περιθώρια βελτιστοποίησης, η τρέχουσα υλοποίηση είναι **αποδεκτή για το παρόν** επειδή η μαζική εισαγωγή rates **εκτελείται μόνο μία φορά** κατά την αρχική σύνδεση με τον Channel Manager. Αντίστοιχα, η μαζική εισαγωγή bookings (βλ. Ενότητα 4.2.2) εκτελείται επίσης μόνο μία φορά κατά την ενεργοποίηση του property. Μετά την αρχική εισαγωγή, οι καθημερινές ενημερώσεις γίνονται μεμονωμένα μέσω webhooks, οπότε η απόδοση είναι ικανοποιητική.

Μελλοντική Βελτιστοποίηση: Για περαιτέρω βελτίωση της απόδοσης, η παράκαμψη των hooks **δεν είναι αρκετή** – απαιτούνται **mass upsert operations** απευθείας στη βάση δεδομένων. Το Payload CMS υποστηρίζει πρόσβαση στο Drizzle ORM μέσω του `payload.db.drizzle`, που επιτρέπει type-safe SQL operations με δυνατότητα batch upserts.

Ωστόσο, αυτή η υλοποίηση **αποφεύγεται προσωρινά** για τους εξής λόγους:

- Κατά τη φάση ανάπτυξης, το schema υφίσταται συχνές αλλαγές (breaking changes).
- Η χρήση low-level SQL/Drizzle queries απαιτεί σταθερό schema για να αποφευχθούν runtime errors.
- Προτιμάται η αναμονή μέχρι το σύστημα να φτάσει σε **stable version** με πραγματικές επιχειρήσεις (businesses).

Μελλοντικά Σενάρια Χρήσης: Η βελτιστοποίηση θα γίνει κρίσιμη σε σενάρια όπου απαιτούνται **τακτικές μαζικές ενημερώσεις**. Ένα τέτοιο σενάριο είναι η ενσωμάτωση με **Revenue Management Systems (RMS)**, τα οποία αλλάζουν αυτόματα τις τιμές βάσει αλγορίθμων ζήτησης. Σε αυτή την περίπτωση, θα χρειαζόταν καθημερινή (π.χ. κάθε βράδυ μέσω cron) ενημέρωση όλων των rates, καθιστώντας την τρέχουσα υλοποίηση ανεπαρκή.

Αντίθετα, τα **bookings** δεν θα απαιτήσουν **αντίστοιχη βελτιστοποίηση**, καθώς η μαζική εισαγωγή γίνεται μόνο μία φορά κατά την ενεργοποίηση, και στη συνέχεια

όλες οι ενημερώσεις (δημιουργία, τροποποίηση, ακύρωση κρατήσεων) λαμβάνονται μέσω **webhooks** σε πραγματικό χρόνο.

[ΔΙΑΓΡΑΜΜΑ: Rate Import Performance Comparison]

Περιγραφή διαγράμματος: Ένα bar chart που συγκρίνει τον χρόνο εκτέλεσης για εισαγωγή 1.000 rates: (1) Με πλήρες lifecycle (~60 seconds), (2) Με direct DB operations (~5 seconds). Επίσης δείχνει τον αριθμό API calls που αποφεύγονται.

Figure 4.5: Σύγκριση απόδοσης εισαγωγής rates.

4.3 Μηχανισμός Auto Actions

Το σύστημα Auto Actions είναι η «καρδιά» του αυτοματισμού της πλατφόρμας.

4.3.1 Κατηγορίες Auto Actions

Υπάρχουν τρεις διακριτοί τύποι:

1. **Notifications to Users:** Ειδοποιήσεις προς τους χρήστες της πλατφόρμας (Managers, Owners, Staff).
2. **Property Notifications to Guests:** Μηνύματα προς τους επισκέπτες (π.χ. οδηγίες άφιξης).
3. **Simple Template:** Πρότυπα κειμένου χωρίς trigger condition, για χειροκίνητη αποστολή μέσω Chat.

4.3.2 Κανάλια Επικοινωνίας

Το σύστημα υποστηρίζει πολλαπλά κανάλια (`preferableChannels`) και προσπαθεί να στείλει το μήνυμα με σειρά προτεραιότητας.

- **Για Guests:** OTA Message (μέσω API καναλιού) ή Email.
- **Για Users:** Telegram. Οι χρήστες μπορούν να κάνουν Subscribe στο Telegram Bot της πλατφόρμας και να λαμβάνουν άμεσες ειδοποιήσεις στο κινητό τους. Η αρχιτεκτονική είναι σχεδιασμένη ώστε να επιτρέπει την εύκολη προσθήκη νέων καναλιών (π.χ. WhatsApp, SMS) στο μέλλον.

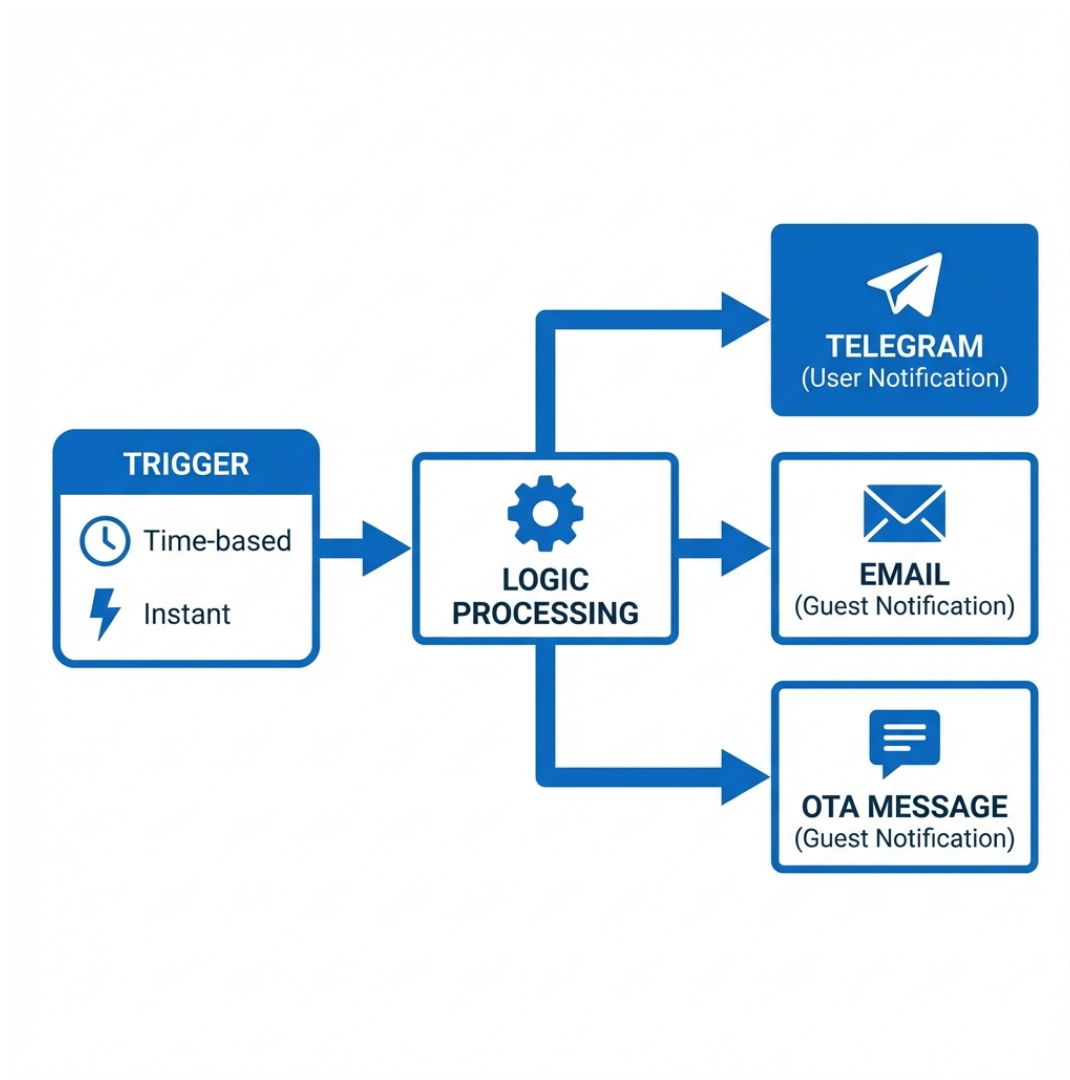


Figure 4.6: Διάγραμμα ροής εκτέλεσης των Auto Actions.

4.3.3 Μηχανισμός Προτύπων (Template Engine)

Αναπτύχθηκε custom Template Engine με τις εξής δυνατότητες:

Variable Replacement: Σύνταξη `[object.property]` για πρόσβαση σε εμφωλευμένα δεδομένα

Conditional Logic: Σύνταξη `{condition?? result}` για εμφάνιση κειμένου υπό συνθήκη

Date Formatting: Αυτόματη αναγνώριση και μορφοποίηση ημερομηνιών

Listing 4.8: Παράδειγμα Template για νέα κράτηση

```
1 Νέακράτηση
2 :□Επιβεβαιωμένηκράτηση
3 - [property.title]□
```

```

4
5 [booking.bookingHolder.firstName] [booking.bookingHolder.lastName]
6 [booking.resource.adults] Ενήλικες, [booking.resource.children]
   Παιδιά
7 [booking.resource.checkIn] έως [booking.resource.checkOut]
8 [property.propertyCountry]
9
10 {user.settings.features.bookings.prices!="none"? Τιμή :
   €[booking.pricing.totalAmount]}
11 {user.settings.features.bookings.prices!="none"? Τιμήανάνύχτα :
   €[booking.resource.pricePerNightνύχτα] / για
   [booking.resource.nights] νύχτες}
12
13 {user.settings.features.bookings.contactDetails!="none"?
   Αριθμόςτηλεφώνου : [booking.bookingHolder.phone]}
14 {user.settings.features.bookings.contactDetails!="none"? Email:
   [booking.bookingHolder.email]}
15
16 {user.settings.features.messages.visibility!="none"? Μηνύματα :
   μπορείτεναδείτεαπαντήσετε / [link-booking-chat]}
17 {user.settings.features.messages.visibility!="none"? Ημερολόγιο :
   [link-booking]}

```

Τεχνική Υλοποίηση: Η υλοποίηση βασίζεται σε Regular Expressions για την αναγνώριση των patterns και αναδρομική διάσχιση των αντικειμένων (booking, property, user) για την εύρεση των τιμών. Ο κώδικας διαχειρίζεται επίσης ειδικές περιπτώσεις, όπως η δημιουργία δυναμικών συνδέσμων ([link-booking]) που οδηγούν σε ασφαλείς σελίδες της πλατφόρμας.

Μελλοντική Επέκταση (GUI): Επί του παρόντος, η σύνταξη των προτύπων γίνεται σε μορφή κειμένου (plain text). Μια σημαντική μελλοντική βελτίωση αφορά την ανάπτυξη ενός γραφικού περιβάλλοντος (GUI Builder), όπου ο χρήστης θα μπορεί να επιλέγει μεταβλητές και συνθήκες από λίστες (drag-and-drop), εξαλείφοντας την ανάγκη απομνημόνευσης της σύνταξης και μειώνοντας την πιθανότητα λαθών.

4.3.4 Time-based Triggers (Cron Jobs)

Αυτές οι ενέργειες εκτελούνται βάσει χρόνου σε σχέση με ένα γεγονός της κράτησης.

- **Trigger Types:** before-checkin, after-checkout, after-cancellation, after-new-booking.

- **Λογική Εκτέλεσης:** Ένα Cron Job τρέχει κάθε ώρα (`everyHourCron.ts`). Για κάθε ενεργό Auto Action, υπολογίζει ένα χρονικό παράθυρο [`fromDate`, `toDate`] βάσει του `timeInterval` και `timeWindow`.
 - *Παράδειγμα:* Για ενέργεια «2 μέρες πριν την άφιξη», το σύστημα ψάχνει κρατήσεις με `checkInDate` που εμπίπτει στο διάστημα [`Τώρα + 48 ώρες`, `Τώρα + 48 ώρες + Window`].
- **Query:** Χρησιμοποιείται το Drizzle/Payload API για να βρεθούν οι κρατήσεις που πληρούν τα κριτήρια και δεν έχουν ήδη λάβει την ειδοποίηση (έλεγχος στο `logging.automationTracking`).

Table 4.2: Υποστηριζόμενοι Time-based Trigger Types

Trigger Type	Περιγραφή	Πεδίο Κράτησης
before-checkin	Πριν την άφιξη	<code>resource.checkInStartOfDay</code>
after-checkout	Μετά την αναχώρηση	<code>resource.checkOutStartOfDay</code>
after-cancellation	Μετά την ακύρωση	<code>cancellationDate</code>
after-new-booking	Μετά τη νέα κράτηση	<code>reservationDate</code>

Listing 4.9: Αλγόριθμος υπολογισμού χρονικού παραθύρου

```

1  const now: Date = new Date();
2  let fromDate: Date;
3  let toDate: Date;
4
5  if (triggerType.includes("before")) {
6    // before: [now + interval, now + interval + window]
7    fromDate = addHours(now, autoAction.timeInterval);
8    toDate = addHours(now, autoAction.timeInterval +
9      autoAction.timeWindow);
10 } else if (triggerType.includes("after")) {
11   // after: [now - interval - window, now - interval]
12   fromDate = subHours(now, autoAction.timeInterval +
13     autoAction.timeWindow);
14   toDate = subHours(now, autoAction.timeInterval);
15 }

```

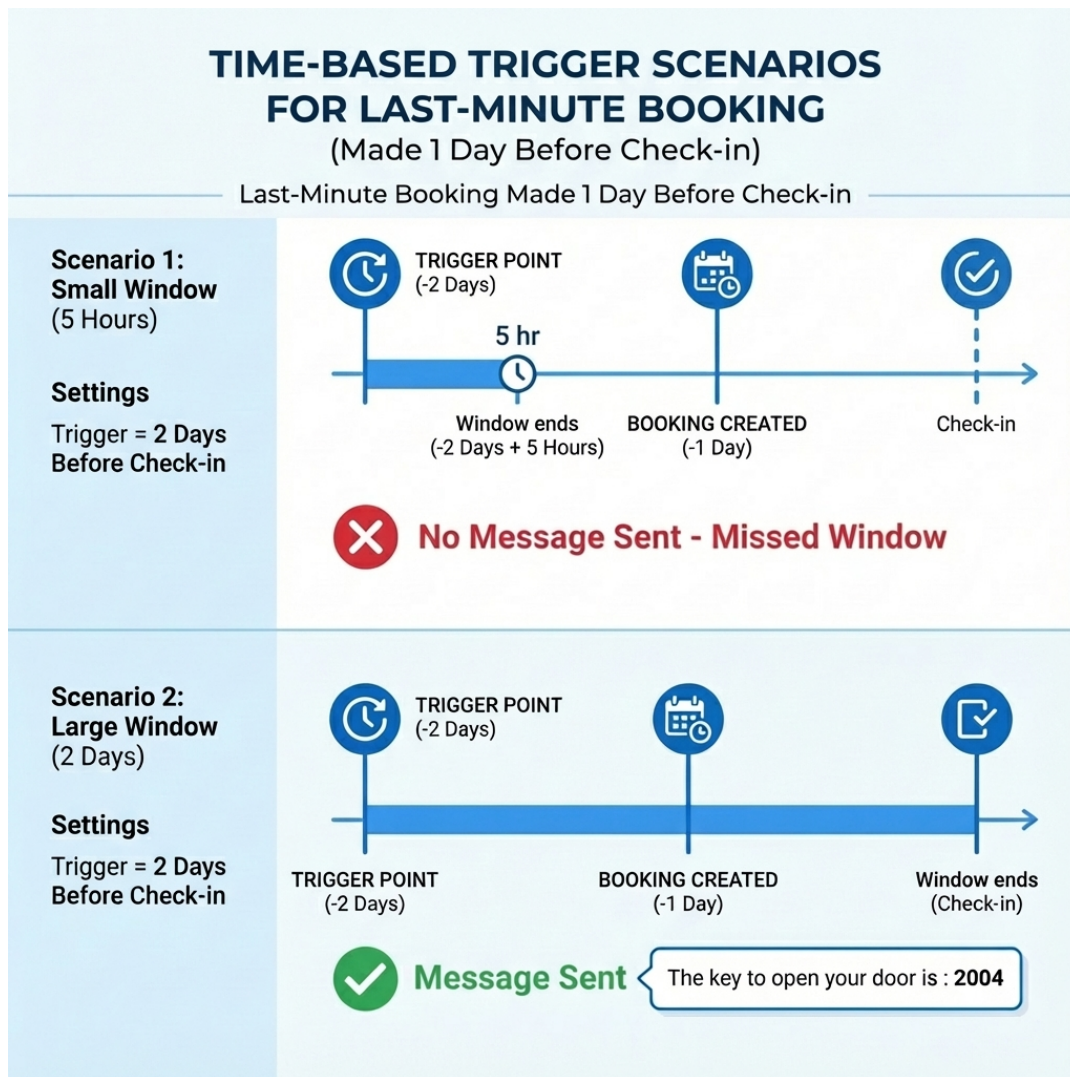


Figure 4.7: Σενάρια λειτουργίας Time-based Triggers για κράτηση της τελευταίας στιγμής (1 μέρα πριν την άφιξη). Στο Σενάριο 1 (μικρό παράθυρο 5 ωρών), η κράτηση χάνει τον κανόνα «2 μέρες πριν» και δεν στέλνεται μήνυμα. Στο Σενάριο 2 (μεγάλο παράθυρο 2 ημερών), η κράτηση εμπίπτει στο παράθυρο και το μήνυμα (π.χ. «The key to open your door is : 2004») αποστέλλεται κανονικά.

4.3.5 Instant Triggers (Webhooks)

Ενέργειες που εκτελούνται άμεσα μόλις συμβεί το γεγονός.

- **Trigger Types:** on-new-message, on-update
- **Υλοποίηση:** Αυτές οι ενέργειες δεν περιμένουν το Cron. Καλούνται απευθείας μέσα από τον Webhook Handler (route.ts) μόλις ολοκληρωθεί επιτυχώς η αποθήκευση της κράτησης/μηνύματος.

4.3.6 Αρχιτεκτονική Cron Jobs

Table 4.3: Scheduled Cron Jobs της πλατφόρμας

Cron Job	Συχνότητα	Αρχείο	Σκοπός
everyHourCron	Κάθε ώρα	everyHourCron.ts	Time-based Auto Actions
everyNightCron	Κάθε βράδυ	everyNightCron.ts	Token refresh, Full sync
everyMonthCron	Αρχή μήνα	everyMonthCron.ts	Billing records

4.4 Σύστημα Χρέωσης και Τιμολόγησης

Η τιμολόγηση και η διαχείριση οικονομικών σχέσεων μεταξύ Platform, Managers και Owners αποτελεί κρίσιμο κομμάτι της πλατφόρμας. Το σύστημα υλοποιεί αυτοματοποιημένη μηνιαία χρέωση μέσω ενός scheduled Cron Job.

4.4.1 Αρχιτεκτονική Χρέωσης

Το σύστημα billing δημιουργεί χρεώσεις ανά χρήστη (Manager ή Owner) στο τέλος κάθε μήνα. Για κάθε μήνα, ένας Owner ή Manager θα έχει το πολύ:

1. **1 Billing όπου είναι Οφειλέτης (Debtor):** Πληρώνει προς την πλατφόρμα (αν είναι Manager) ή προς τον Manager (αν είναι Owner).
2. **1 Billing όπου είναι Πιστωτής (Creditor)** (εφόσον είναι Manager): Λαμβάνει πληρωμές από τους Owners που διαχειρίζεται.

Επιπλέον, τα Billing records δημιουργούνται όταν υπάρχουν **unallocated bookings** – κρατήσεις που δεν έχουν ακόμη αντιστοιχιστεί σε κάποιο Billing record. Κατά τη διαδικασία μηνιαίας χρέωσης, το σύστημα αναζητά bookings της περιόδου που δεν έχουν συμπεριληφθεί σε κανένα bill και τα συσχετίζει με το νεοδημιουργούμενο Billing.

- **Owner Bill:** Ο Owner (Debtor) οφείλει στον Manager (Creditor). Περιλαμβάνει commissions και fixed fees.
- **Manager Bill:** Ο Manager (Debtor) οφείλει στην Πλατφόρμα (Creditor). Περιλαμβάνει platform commissions.

4.4.2 Αυτόματη Μηνιαία Δημιουργία Bills (Cron Job)

Στην αρχή κάθε μήνα, εκτελείται αυτόματα το everyMonthCron που δημιουργεί τα billing records για τον προηγούμενο μήνα. Η διαδικασία περιλαμβάνει την εύρεση

όλων των Managers και Owners με τα αντίστοιχα properties τους, τον υπολογισμό της περιόδου χρέωσης (αρχή και τέλος μήνα, ημερομηνία λήξης πληρωμής), και τη δημιουργία του κατάλληλου billing record ανά χρήστη. Όλες οι λειτουργίες εκτελούνται εντός ενός database transaction για να εξασφαλιστεί η ατομικότητα των ενεργειών.

4.4.3 Ανάλυση Χρεώσεων (Pricing Breakdown)

Κάθε bill περιλαμβάνει λεπτομερή ανάλυση των χρεώσεων:

Table 4.4: Τύποι χρεώσεων Billing

Τύπος	Περιγραφή	Υπολογισμός
propertyCommission	Χρέωση ανά ακίνητο	$\text{count} \times \text{pricePerProperty}$
unitCommission	Χρέωση ανά δωμάτιο/μονάδα	$\text{count} \times \text{pricePerUnit}$
bookingsCommission	Προμήθεια κρατήσεων	Άθροισμα προμηθειών όλων των κρατήσεων

Όταν ο χρήστης πατάει το «Recalculate» button, το `billingBeforeChangeHook` επανυπολογίζει τις χρεώσεις, αναζητώντας όλες τις κρατήσεις της περιόδου και αθροίζοντας τις αντίστοιχες προμήθειες.

4.4.4 Προβολή Bills ανά Ρόλο Χρήστη

Το σύστημα access control εξασφαλίζει ότι κάθε χρήστης βλέπει μόνο τα σχετικά bills:

Table 4.5: Προβολή Billing ανά Ρόλο

Ρόλος Χρήστη	Τι Βλέπει
Platform User	Όλα τα billing records
Manager	Τα δικά του manager-bills (τι οφείλει στην πλατφόρμα) + Τα owner-bills όπου είναι creditor (τι του οφείλουν οι Owners)
Owner	Τα δικά του owner-bills (τι οφείλει στον Manager του)
Staff	Τα owner-bills του Owner που ανήκουν (read-only)

4.4.5 Κατάσταση Πληρωμής

Κάθε bill έχει μια `payment.status` που μπορεί να είναι:

- `unpaid`: Δεν έχει πληρωθεί
- `partially`: Μερική πληρωμή

- `paid-direct`: Πληρώθηκε απευθείας (μετρητά, τραπεζική μεταφορά)
- `paid-online`: Πληρώθηκε online μέσω της πλατφόρμας

[ΔΙΑΓΡΑΜΜΑ: Billing UI Screenshots]

Περιγραφή διαγράμματος: Δύο screenshots που δείχνουν: (1) Την προβολή του Manager με τα `owner-bills` που περιμένει να εισπράξει και τα `manager-bills` που πρέπει να πληρώσει, (2) Την προβολή του Owner με τα bills που οφείλει στον Manager του, μαζί με breakdown των χρεώσεων.

Figure 4.8: Διεπαφή χρήστη Billing.

4.4.6 Σύστημα Ελέγχου Πρόσβασης (Access Control System)

Η πλατφόρμα υλοποιεί ένα εξελιγμένο σύστημα ελέγχου πρόσβασης που βασίζεται σε δύο άξονες:

1. **Ρόλος Χρήστη (User Type)**: `platform-user`, `manager`, `owner`, `staff`
2. **Χαρακτηριστικά Πλάνου (Plan Features)**: Ρυθμίσεις που καθορίζονται από τον Manager για κάθε Owner/Staff

Plan Features Configuration: Ο Manager μπορεί να ρυθμίσει τα δικαιώματα για κάθε Owner ή Staff μέσω του πεδίου `settings.features`. Τα διαθέσιμα features αφορούν τα Bookings (περιεχόμενο, τιμές, στοιχεία επικοινωνίας), τα Messages (ορατότητα και δυνατότητα απάντησης), το Property (περιεχόμενο και rates), και το Billing (ορατότητα). Κάθε feature μπορεί να έχει τιμή `none` (καμία πρόσβαση), `view` (μόνο ανάγνωση), ή `write` (πλήρης πρόσβαση).

Table 4.6: Πίνακας Δικαιωμάτων

Πόρος	Feature Setting	Platform User	Manager	Owner	Staff
Bookings - Βασικά	<code>content: view</code>	☐ All	☐ Managed Properties	☐ Owned Properties	☐ Owner's Properties
Bookings - Βασικά	<code>content: write</code>	☐	☐	☐ Edit own	×
Bookings - Τιμές	<code>prices: view</code>	☐	☐	<i>If allowed</i>	<i>If allowed</i>
Bookings - Επικ.	<code>contactDetails: view</code>	☐	☐	<i>If allowed</i>	<i>If allowed</i>
Messages	<code>visibility: write</code>	☐	☐	<i>If allowed</i>	<i>If allowed</i>
Billing	<code>visibility: view</code>	☐	☐ Own bills	<i>If allowed</i>	×

Πίνακας Δικαιωμάτων ανά Συνδυασμό:

Υλοποίηση Access Functions: Κάθε collection διαθέτει access functions που αξιολογούν τον συνδυασμό ρόλου και features. Για παράδειγμα, οι Platform Users έχουν πρόσβαση σε όλα τα δεδομένα, οι Managers βλέπουν μόνο τα bookings των ακινήτων που διαχειρίζονται, οι Owners βλέπουν τα δικά τους ακίνητα, και το Staff ακολουθεί τους περιορισμούς του Owner στον οποίο ανήκει με επιπλέον restrictions.

Field-Level Access Control: Επιπλέον του collection-level access, υπάρχει και field-level control. Κάθε πεδίο μπορεί να έχει τις δικές του access functions για read και update, επιτρέποντας την απόκρυψη ευαίσθητων πληροφοριών (π.χ. τιμές, στοιχεία επικοινωνίας) ακόμη και αν ο χρήστης έχει πρόσβαση στο collection.

Παράδειγμα Σεναρίου: Ένας Manager δημιουργεί έναν χρήστη «Καθαριστής» (Staff) με τα εξής δικαιώματα:

- `bookings.content`: 'view' – Μπορεί να δει τις κρατήσεις
- `bookings.prices`: 'none' – Δεν μπορεί να δει τιμές
- `bookings.contactDetails`: 'none' – Δεν μπορεί να δει στοιχεία επικοινωνίας
- `messages.visibility`: 'none' – Δεν μπορεί να δει μηνύματα

Όταν ο Καθαριστής ανοίξει μια κράτηση:

1. Βλέπει: Ημερομηνίες check-in/out, Property name, Room name
2. ΔΕΝ βλέπει: Τιμή, Email πελάτη, Τηλέφωνο, Μηνύματα

Access Control Decision Tree

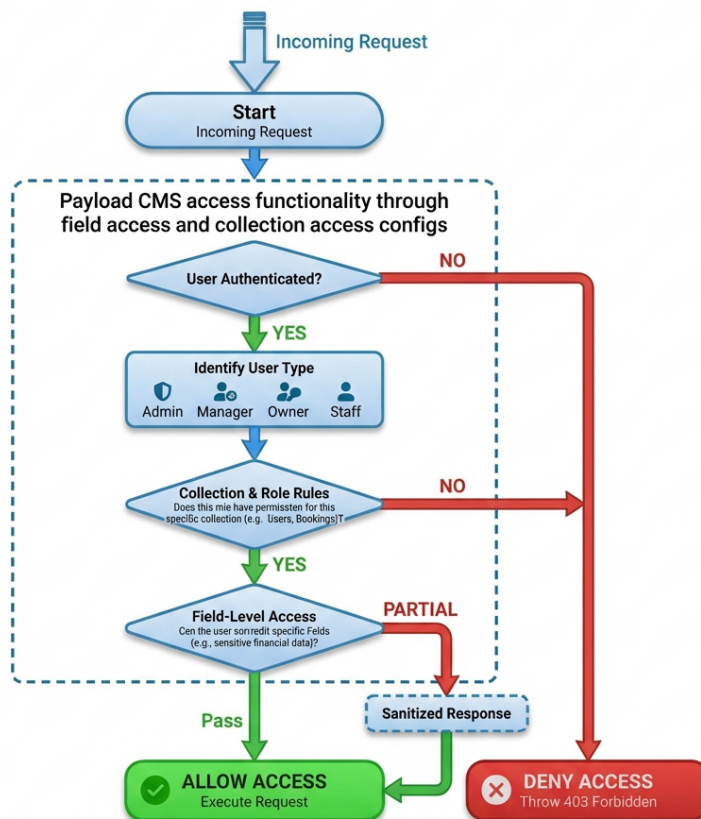


Figure 4.9: Decision tree για τον έλεγχο πρόσβασης.

4.5 Αυτοματοποιημένη Παραγωγή Ιστοσελίδων

Η πλατφόρμα παρέχει τη δυνατότητα αυτόματης παραγωγής ιστοσελίδων για καταλύματα, αποτελώντας μια ολοκληρωμένη λύση direct booking.

4.5.1 Δομή και Λειτουργία Website

Το παραγόμενο website ακολουθεί βελτιστοποιημένη δομή τριών κύριων σελίδων:

[INFOGRAPHIC: Website Page Architecture]

Περιγραφή: Τρεις κάθετες στήλες που αναπαριστούν τις σελίδες: (1) Home Page «/» - με Multi-room search form, Property cards grid, Filter options (2) Property Page «/properties/[id]» - με Gallery, Description, Room list, AI Chat Bubble, Book Now CTA (3) Booking Page «/booking/[id]» - με Booking details, Payment info, Guest self-service options. Βέλη δείχνουν τη ροή πλοήγησης του χρήστη.

Figure 4.10: Αρχιτεκτονική σελίδων Website.

1. **Home Page (/):** Κεντρική σελίδα με Multi-room search
 - Επιλογή ημερομηνιών check-in/check-out
 - Επιλογή αριθμού ενηλίκων/παιδιών
 - Real-time availability filtering
2. **Property Page (/properties/[propertyId]):** Σελίδα λεπτομερειών καταλύματος
 - Gallery με εικόνες
 - Περιγραφή και παροχές
 - AI Chat Bubble για ερωτήσεις
 - Booking form
3. **Booking Management (/booking/[bookingId]):** Σελίδα διαχείρισης κράτησης για τον επισκέπτη
 - Προβολή λεπτομερειών κράτησης
 - Δυνατότητα ακύρωσης (αν επιτρέπεται)
 - Στοιχεία επικοινωνίας

4.5.2 Website API Endpoints

Το website επικοινωνεί με την πλατφόρμα μέσω secure API endpoints:

[INFOGRAPHIC: Website API Flow]

Περιγραφή: Sequence diagram που δείχνει: Website → Platform API:
"/api/website/properties" (GET) → ... → Booking confirmation.

Figure 4.11: Ροή επικοινωνίας Website API.

Table 4.7: Website API Endpoints

Endpoint	Method	Σκοπός
/api/website/properties	GET	Λίστα καταλυμάτων του website
/api/website/properties/[id]	GET	Λεπτομέρειες ενός καταλύματος
/api/website/availability	POST	Έλεγχος διαθεσιμότητας για ημερομηνίες
/api/website/booking	POST	Δημιουργία κράτησης
/api/website/booking/[id]	GET	Λεπτομέρειες κράτησης

Αλγόριθμος Ελέγχου Διαθεσιμότητας

Listing 4.10: Έλεγχος Διαθεσιμότητας

```

1 // Endpoint: /api/website/availability
2 export const getAvailabilities = async (
3   websiteApiKey: string,
4   checkIn: number,
5   checkOut: number,
6   rooms: { adults: number; children: number }[]
7 ) => {
8   // 1. Βρες τα properties που ανήκουν σε αυτό το website
9   const website = await payload.find({
10     collection: "website",
11     where: { apiKey: { equals: websiteApiKey } },
12   });
13
14   // 2. Φέρει τα rates για τις ζητούμενες ημερομηνίες
15   const rates = await payload.find({
16     collection: "property-rates",
17     where: {
18       property: { in: website.properties },
19       date: {
20         greater_than_equal: checkIn,
21         less_than: checkOut,
22       },

```

```
23     },
24   });
25
26   // 3. Φιλτράρισμα βάσει availUnits > 0
27   const availableProperties = rates.docs.filter(
28     (rate) => rate.availability.availUnits > 0
29   );
30
31   // 4. Matching rooms με ζητούμενη χωρητικότητα
32   return matchRoomsToRequirements(availableProperties, rooms);
33 };
```

4.5.3 Deploy URL & Vercel Integration

Η διαδικασία «One-Click Deploy» απλοποιεί τη δημιουργία website:

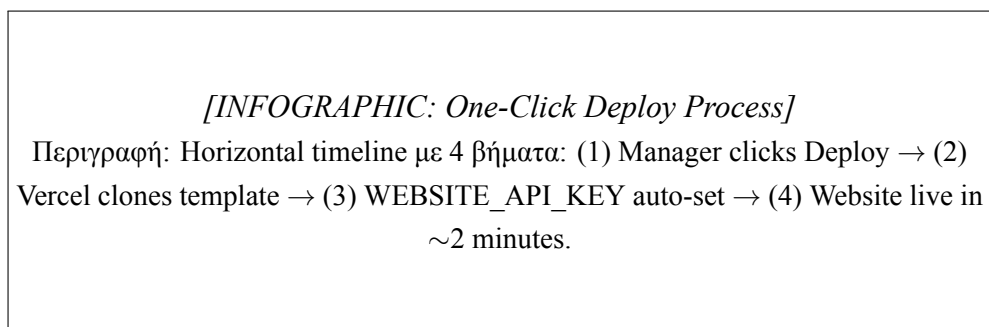


Figure 4.12: Διαδικασία One-Click Deploy.

1. Ο Manager κλικάρει το «Deploy to Vercel» button
2. Το Vercel κλωνοποιεί το public repo lunarety-v2-website
3. Το WEBSITE_API_KEY τίθεται αυτόματα ως environment variable
4. Το website είναι live μέσα σε ~2 λεπτά

4.5.4 Website Booking Creation Flow

Όταν ένας επισκέπτης κάνει κράτηση μέσω του website:

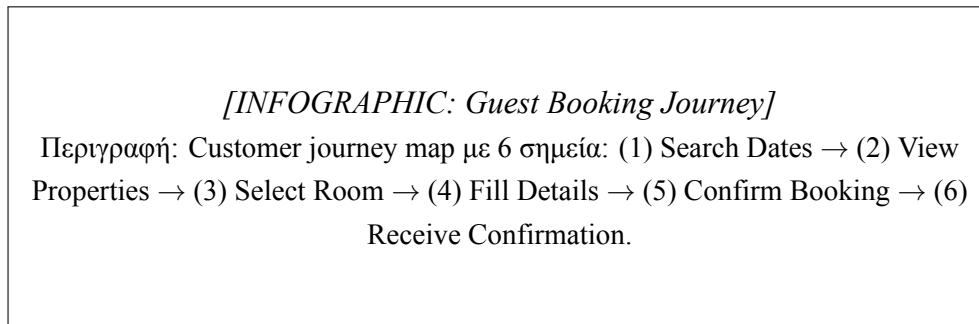


Figure 4.13: Ταξίδι κράτησης επισκέπτη.

Listing 4.11: Δημιουργία κράτησης από Website

```
1 // Website booking creation
2 const createWebsiteBooking = async (bookingData, websiteApiKey) => {
3   // Δημιουργία κράτησης με source = "website"
4   const booking = await payload.create({
5     collection: "bookings",
6     data: {
7       ...bookingData,
8       source: "website", // Σημειώνει ότι ήρθε από website
9       website: websiteId, // Reference στο website
10    },
11  });
12
13  // To lifecycle hook αναλαμβάνει:
14  // 1. Commission calculation
15  // 2. Sync με Beds24
16  // 3. Rate linking
17  // 4. Auto-actions triggering
18
19  return booking;
20 };
```

4.5.5 Επιχειρηματικές Ευκαιρίες (Platform as OTA)

Η ευελιξία του συστήματος δημιουργεί νέες επιχειρηματικές ευκαιρίες:

[INFOGRAPHIC: Platform as OTA Business Model]

Περιγραφή: Διάγραμμα που δείχνει δύο μοντέλα: (1) Traditional - Guest → Airbnb/Booking (15-20% fee) → Property. (2) Platform as OTA - Guest → Manager's Platform Website (5% fee) → Property.

Figure 4.14: Επιχειρηματικό μοντέλο Platform as OTA.

- **Owner Website:** Ένα website για κάθε Owner με τα δικά του ακίνητα
- **Platform Website:** Ένα συγκεντρωτικό website με όλα τα ακίνητα του Manager
- **Hybrid Model:** Ο Manager λειτουργεί ως μικρό OTA, κρατώντας προμήθεια από τους Owners για τις απευθείας κρατήσεις

4.6 Περιβάλλον Χρήστη (Platform UI)

4.6.1 Dashboard και Calendar

Το ημερολόγιο δεν υποστηρίζει drag-and-drop για λόγους ακρίβειας. Αντ' αυτού, προσφέρει εργαλεία μαζικής επεξεργασίας:

- **Επιλογή Κρατήσεων:** Ο χρήστης επιλέγει μια κράτηση για να δει λεπτομέρειες ή να την επεξεργαστεί.
- **Επιλογή Ημερομηνιών:** Ο χρήστης επιλέγει ένα εύρος ημερομηνιών και μπορεί να αλλάξει μαζικά τιμές και διαθεσιμότητα.
- **Φίλτρα:** Δυνατότητα επιλογής συγκεκριμένων καταλυμάτων για προβολή.

4.6.2 Σύστημα Μηνυμάτων

Χρήστες με τα κατάλληλα δικαιώματα μπορούν να απαντήσουν σε φιλοξενούμενους απευθείας στα συνδεδεμένα OTAs μέσω της διεπαφής συνομιλίας. Στη σελίδα /messages, παρέχεται πρόσβαση σε όλα τα πρόσφατα μηνύματα, τα οποία επιστρέφονται ταξινομημένα από το νεότερο στο παλαιότερο και υποστηρίζουν πλήρη σελιδοποίηση με infinite scrolling. Ο χρήστης μπορεί να αναζητήσει συνομιλίες χρησιμοποιώντας φίλτρα όπως το όνομα κράτησης, το κατάλυμα ή το κείμενο του τελευταίου μηνύματος. Επιπλέον, το Chat επιτρέπει τη χρήση των **Simple Templates** (που συμπληρώνονται

αυτόματα με στοιχεία της κράτησης) και τη βελτίωση κειμένου μέσω AI («Improve with AI»).

4.6.3 Διαχείριση Δικαιωμάτων και Drawers

Η επεξεργασία γίνεται μέσω UI Drawers με δυναμικό περιεχόμενο βάσει δικαιωμάτων.

Παράδειγμα: Ένας Manager μπορεί να ορίσει ότι ο χρήστης «Καθαριστής» (Staff) μπορεί να βλέπει τις κρατήσεις αλλά δεν μπορεί να δει τα στοιχεία επικοινωνίας του πελάτη ούτε να αλλάξει τις ημερομηνίες. Το Drawer θα προσαρμοστεί αυτόματα, κρύβοντας ή απενεργοποιώντας τα αντίστοιχα πεδία.

Στρατηγικές Ενημέρωσης (Update Strategies):

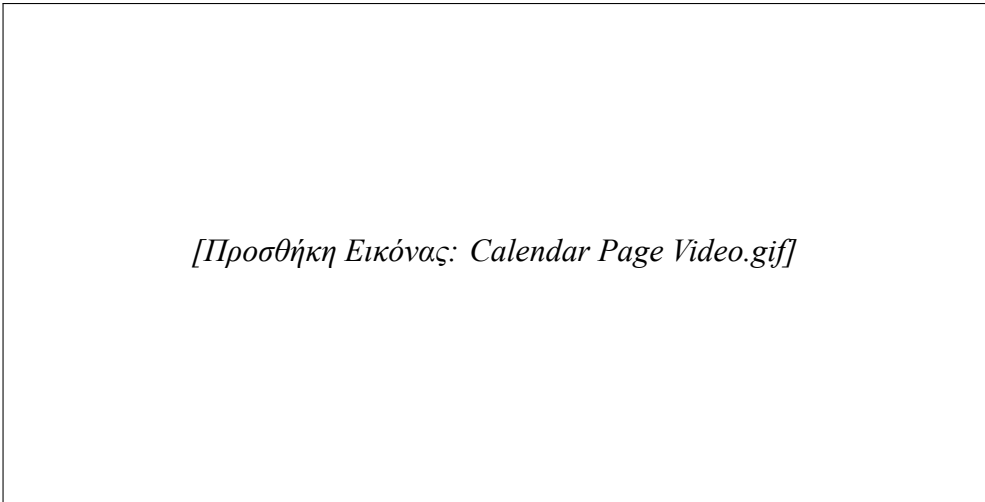
- **Booking Update:** Ενεργοποιεί το πλήρες lifecycle (επικοινωνία με Channel Manager → update/create booking).
- **Availability Update:** Για να αποφευχθεί ο φόρτος στο Channel Manager (ένα request ανά ημέρα), ακολουθείται άλλη στρατηγική. Γίνεται πρώτα μαζικό update στο Channel Manager και, εφόσον επιτύχει, ενημερώνονται απευθείας τα Rates στη βάση δεδομένων (Direct DB Update), παρακάμπτοντας τα hooks για ταχύτητα.

Chapter 5

Αποτελέσματα

5.1 Παρουσίαση της Πλατφόρμας

Η πλατφόρμα προσφέρει ένα μοντέρνο, responsive περιβάλλον εργασίας. Στα παρακάτω οπτικά παραδείγματα (GIFs) παρουσιάζεται η οπτική γωνία ενός χρήστη με πλήρη δικαιώματα (Admin/Manager). Είναι σημαντικό να τονιστεί ότι χρήστες με διαφορετικούς ρόλους (όπως Owners ή Staff) ενδέχεται να μην έχουν τη δυνατότητα επεξεργασίας ή ακόμη και προβολής όλων των τμημάτων που απεικονίζονται, ανάλογα με τα δικαιώματα που τους έχουν εκχωρηθεί.



[Προσθήκη Εικόνας: Calendar Page Video.gif]

Figure 5.1: Η σελίδα του Ημερολογίου (Calendar). Το αρχικό ημερολόγιο έχει σχεδιαστεί ως ένα «Easy Calendar». Για τον βασικό χρήστη, λειτουργεί ως μια απλή λίστα καθηκόντων που δείχνει τι πρέπει να γίνει κάθε μέρα (αφίξεις, αναχωρήσεις), καθιστώντας το εξαιρετικά εύχρηστο για την καθημερινότητα. Ωστόσο, προσφέρει τη δυνατότητα προσθήκης περισσότερης λειτουργικότητας για πιο απαιτητικούς χρήστες, χωρίς να θυσιάζεται η ευκολία χρήσης.

[Προσθήκη Εικόνας: Messages Page Video.gif]

Figure 5.2: Η σελίδα των Μηνυμάτων. Το σύστημα μηνυμάτων παρέχει δυνατότητες άμεσης επικοινωνίας, υποστηρίζοντας τη χρήση προτύπων (templates) και τη βελτίωση κειμένου μέσω ΑΙ.

[Προσθήκη Εικόνας: USERS (advanced dashboard).gif]

Figure 5.3: Διαχείριση Χρηστών (Users) στο Advanced Dashboard.

[Προσθήκη Εικόνας: PLANS (advanced dashboard).gif]

Figure 5.4: Διαχείριση Πλάνων Χρέωσης (Plans) στο Advanced Dashboard.

[Προσθήκη Εικόνας: AUTO_ACTIONS (advanced dashboard).gif]

Figure 5.5: Διαχείριση Αυτοματισμών (Auto Actions) στο Advanced Dashboard.

5.2 Παρουσίαση του Generated Website

Η πλατφόρμα παρέχει τη δυνατότητα αυτόματης παραγωγής ιστοσελίδων για τα καταλύματα, προσφέροντας μια ολοκληρωμένη εμπειρία τόσο για τον διαχειριστή όσο και για τον επισκέπτη.

[Προσθήκη Εικόνας: Website Generation (Admin).gif]

Figure 5.6: Διαδικασία παραγωγής Website από το περιβάλλον του διαχειριστή. Ο διαχειριστής μπορεί με απλά βήματα να δημιουργήσει και να δημοσιεύσει την ιστοσελίδα του καταλύματος, αξιοποιώντας τα δεδομένα που είναι ήδη καταχωρημένα στην πλατφόρμα.

[Προσθήκη Εικόνας: Website Demo (Navigate inside Website like a guest).gif]

Figure 5.7: Πλοήγηση επισκέπτη στην παραγόμενη ιστοσελίδα. Η ιστοσελίδα που παράγεται είναι πλήρως λειτουργική, επιτρέποντας στον επισκέπτη να δει λεπτομέρειες του καταλύματος, φωτογραφίες και παροχές, καθώς και να προχωρήσει σε κράτηση.

[Προσθήκη Εικόνας: Website Demo (LLM and AI capabilities that guests can use).gif]

Figure 5.8: Δυνατότητες AI και LLM για την εξυπηρέτηση επισκεπτών. Ενσωματωμένες δυνατότητες Τεχνητής Νοημοσύνης επιτρέπουν στους επισκέπτες να αλληλεπιδρούν με έξυπνους βοηθούς για να λαμβάνουν άμεσες απαντήσεις σε ερωτήματα σχετικά με το κατάλυμα και την περιοχή.

5.3 Smart Features & LLM Integration

Η ενσωμάτωση LLMs προσφέρει απτά οφέλη:

- **Smart Chat (Platform):** Στο περιβάλλον μηνυμάτων, το AI προτείνει απαντήσεις λαμβάνοντας υπόψη το ιστορικό της συνομιλίας και τις πληροφορίες της κράτησης.
- **Website AI Assistant (OpenRouter):** Στις ιστοσελίδες των καταλυμάτων (`/properties/[proper` υπάρχει η δυνατότητα ενεργοποίησης ενός **AI Chat Bubble**.
 - **Configuration:** Ο Manager μπορεί να εισάγει ένα `OpenRouter API key` στις ρυθμίσεις του Website.
 - **Context-Aware:** Το Chat Bubble είναι «Fixed» στη γωνία της οθόνης. Γνωρίζει τα πάντα για το κατάλυμα (παροχές, τοποθεσία) αλλά και δυναμικά δεδομένα όπως **τρέχουσα διαθεσιμότητα και τιμές**. Έτσι, μπορεί να απαντήσει σε ερωτήσεις τύπου «Είναι ελεύθερο το σπίτι για το επόμενο Σαββατοκύριακο και πόσο κοστίζει;» με ακρίβεια, λειτουργώντας ως ένας 24/7 ψηφιακός ρεσεψιονίστ.

5.4 Τεχνικές Προκλήσεις και Λύσεις

5.4.1 Συγχρονισμός Δεδομένων με External APIs

Πρόβλημα: Διατήρηση συνέπειας μεταξύ τοπικής βάσης και Beds24.

Λύση: Υιοθέτηση στρατηγικής όπου το Beds24 είναι η κύρια πηγή αλήθειας για δεδομένα κρατήσεων, με merge logic για internal fields.

5.4.2 Αποφυγή Infinite Loops

Πρόβλημα: Two-way sync δημιουργεί κύκλους.

Λύση: Χρήση skipChannelSync flag για αποτροπή reverse-update.

5.4.3 Απόδοση Μαζικών Εισαγωγών

Πρόβλημα: Εισαγωγή ~60.000 records με standard lifecycle απαιτεί ώρες.

Λύση: Hybrid strategy με direct DB operations για bulk, standard lifecycle για μεμονωμένες αλλαγές.

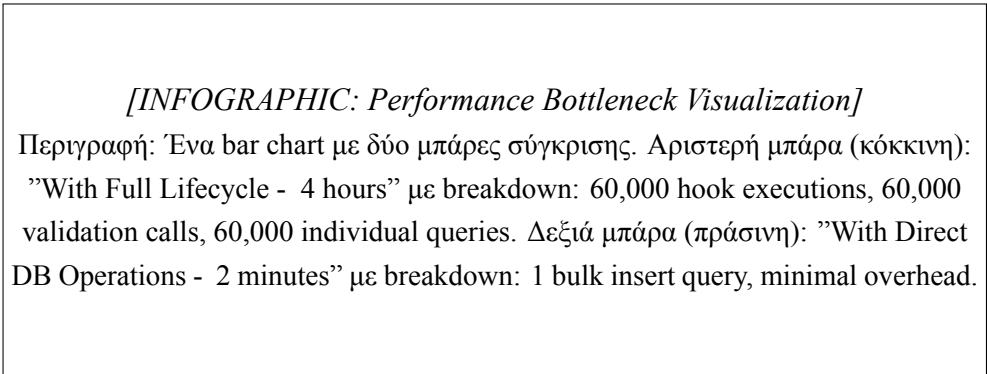


Figure 5.9: Σύγκριση απόδοσης Import.

Table 5.1: Σύγκριση στρατηγικών εισαγωγής

Περίπτωση	Μέθοδος	Πλεονέκτημα
Mass Property Import	Direct DB (<code>payload.db.create</code>)	50× ταχύτερο
Mass Rate Import	Bulk Insert	1 query για χιλιάδες records
Individual Updates	Standard Lifecycle	Consistency, hooks execute

Listing 5.1: Mass import με direct DB

```

1 // Mass import με direct DB operations
2 const insertPropertyRates = async (rates: PropertyRate[]) => {
3   // Χρήση Drizzle για bulk insert
4   await db.insert(propertyRatesTable).values(
5     rates.map((rate) => ({
6       date: rate.date,
7       property: rate.property,
8       channelRoomId: rate.channelRoomId,
9       availabilityStatus: rate.availability.status,
10      price: rate.availability.price,
11      minNights: rate.availability.minNights,
12      maxNights: rate.availability.maxNights,
13    })))
14  };
15 };

```

5.4.4 Διαχείριση Σύνθετων Rate Structures

Πρόβλημα: Το Beds24 στέλνει τα rates σε συμπυκνμένη μορφή (date ranges), ενώ η πλατφόρμα χρειάζεται ημερήσια granularity για:

- Υπολογισμό διαθεσιμότητας
- Σύνδεση με κρατήσεις
- Υπολογισμό τιμών

[INFOGRAPHIC: Rate Data Transformation]

Περιγραφή: Διάγραμμα μετατροπής δεδομένων. Αριστερά: "Beds24 Format" με ένα JSON object {from: "2025-01-01", to: "2025-01-31", price: 100}. Βέλος με ετικέτα "Expand to daily records". Δεξιά: "Platform Format" με 31 μικρά κελιά, κάθε ένα αναπαριστώντας μία ημέρα με τιμή €100.

Figure 5.10: Μετατροπή δομής Rate.

Λύση - Date Range Expansion Algorithm:

Listing 5.2: Επέκταση date range

```

1 // Επέκταση range σε μεμονωμένες ημέρες
2 const expandDateRange = (range: DateRange): DailyRate[] => {
3   const days = eachDayOfInterval({

```

```

4   start: new Date(range.from),
5   end: new Date(range.to),
6   });
7
8   return days.map((day) => ({
9     date: Number(format(day, "yyyyMMdd")), // YYYYMMDD format
10    price: range.price,
11    minNights: range.minStay ?? 1,
12    maxNights: range.maxStay ?? 365,
13    status: range.override ?? "none",
14  }));
15 };

```

5.4.5 Multi-Tenant Data Isolation

Πρόβλημα: Η πλατφόρμα εξυπηρετεί πολλούς Managers, καθένας με τους δικούς του Owners, Properties, και Bookings. Η απομόνωση δεδομένων είναι κρίσιμη:

- Ο Manager A δεν πρέπει ποτέ να δει δεδομένα του Manager B
- Ο Owner πρέπει να βλέπει μόνο τα δικά του properties

[INFOGRAPHIC: Multi-Tenant Data Isolation]

Περιγραφή: Τρεις κάθετες στήλες που αναπαριστούν τρεις Managers (A, B, C). Κάθε στήλη περιέχει "bubble" για τα δικά τους δεδομένα: Properties, Owners, Bookings. Μεταξύ των στηλών υπάρχει μια διακεκομμένη κόκκινη γραμμή με ετικέτα "Isolation Boundary - No Cross-Access".

Figure 5.11: Απομόνωση δεδομένων Multi-Tenant.

Λύση - Query Filtering at Access Layer:

Listing 5.3: Query Filtering

```

1  // Κάθε collection έχει access functions που φιλτράρουν βάσει ιεραρχίας
2  export const readAccess = async ({ req }: { req: PayloadRequest }) => {
3    const user = req.user as User;
4
5    // Platform users: no filter
6    if (user.type === "platform-user") return true;
7
8    // Managers: only their properties

```

```
9   if (user.type === "manager") {
10     return { manager: { equals: user.id } };
11   }
12
13   // Owners: only properties assigned to them
14   if (user.type === "owner") {
15     return { owner: { equals: user.id } };
16   }
17
18   // Staff: properties of their parent owner/manager
19   if (user.type === "stuff") {
20     const parentId = user.ownedBy ?? user.managedBy;
21     return { owner: { equals: parentId } };
22   }
23
24   return false; // Deny by default
25 };
```

5.4.6 Time-Zone Management

Πρόβλημα: Οι κρατήσεις μπορεί να γίνονται από επισκέπτες σε διαφορετικές ζώνες ώρας, αλλά το check-in/check-out αναφέρεται στην τοπική ώρα του καταλύματος.

[INFOGRAPHIC: Timezone Confusion Example]

Περιγραφή: Ένας κόσμος με τρεις τοποθεσίες επισημασμένες: New York (UTC-5), London (UTC), Athens (UTC+2). Βέλη δείχνουν: "Guest books at 11 PM New York time" → "Property's check-in is 3 PM Athens time next day" → "When should auto-action trigger?"

Figure 5.12: Διαχείριση ζωνών ώρας.

Λύση - UTC Normalization + Property Timezone:

Listing 5.4: Υπολογισμός Trigger Time

```
1 // Αποθήκευση πάντα σε UTC, μετατροπή βάσει property timezone
2 const calculateTriggerTime = (booking, autoAction) => {
3   const propertyTimezone = booking.property.timezone ?? "Europe/Athens";
4
5   // Υπολογισμός check-in στην τοπική ώρα
6   const checkInLocal = formatInTimeZone(
```

```

7   new Date(booking.resource.checkInStartOfDay),
8   propertyTimezone,
9   "yyyy-MM-dd'T'HH:mm:ss"
10  );
11
12  // Εφαρμογή του trigger offset
13  const triggerTime = addHours(checkInLocal, -autoAction.timeInterval);
14
15  // Μετατροπή πίσω σε UTC για αποθήκευση
16  return toZonedTime(triggerTime, "UTC");
17 };

```

5.5 Στατιστικά και Μετρήσεις Κώδικα

Η πλατφόρμα αποτελείται από σημαντικό volume κώδικα, οργανισμένο σε διακριτά modules:

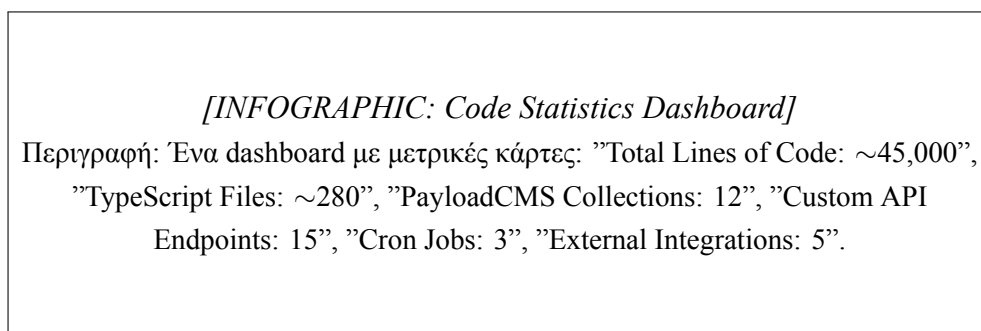


Figure 5.13: Στατιστικά Κώδικα.

Table 5.2: Στατιστικά project

Κατηγορία	Αριθμός	Σημειώσεις
PayloadCMS Collections	12	Users, Bookings, Properties, etc.
Custom Hooks	~35	beforeChange, afterChange
API Endpoints	15	Webhooks, Website API, Auth
Cron Jobs	3	Hourly, Nightly, Monthly
React Components	~50	Calendar, Chat, Drawers
Utility Functions	~100	Date handling, formatting

Chapter 6

Συζήτηση

6.1 Αξιολόγηση Τεχνολογικών Επιλογών

Η υιοθέτηση του μοντέλου «Backend Framework» με το PayloadCMS v3 και η χρήση Serverless υποδομών απέδειξε ότι είναι δυνατή η ανάπτυξη πολύπλοκων εφαρμογών με ελάχιστους πόρους (Rapid Development). Η επιλογή της PostgreSQL έναντι της MongoDB δικαιώθηκε από την ανάγκη για αυστηρές σχέσεις μεταξύ των οντοτήτων (Manager-Owner-Property). Η μεγαλύτερη πρόκληση ήταν η διαχείριση της ασύγχρονης φύσης των Webhooks και η διασφάλιση της συνοχής των δεδομένων, η οποία αντιμετωπίστηκε επιτυχώς με τη χρήση transactions.

Ένα σημείο που αναγνωρίζεται ως πεδίο μελλοντικής βελτιστοποίησης είναι η εμπειρία χρήστη (UX) στις περιοχές διαμόρφωσης του συστήματος, και συγκεκριμένα στη διαχείριση Χρηστών (Users), Αυτοματισμών (Auto Actions) και Πλάνων (Plans). Επί του παρόντος, αυτές οι λειτουργίες βασίζονται στο default στυλ του Payload CMS, το οποίο παρουσιάζει μια πιο σύνθετη εμπειρία χρήσης σε σχέση με το υπόλοιπο custom UI της πλατφόρμας. Η επιλογή αυτή έγινε με γνώμονα την εστίαση στην τελειοποίηση των εργαλείων καθημερινής διαχείρισης (ξενοδοχεία και διαμερίσματα), τα οποία αποτελούν τον πυρήνα της δραστηριότητας. Οι ρυθμίσεις Χρηστών, Πλάνων και Αυτοματισμών ενημερώνονται σπάνια και κυρίως από εκπαιδευμένους Managers, και λιγότερο από τους Owners, οι οποίοι επιζητούν μια έτοιμη λύση («out-of-the-box») που θα διαχειρίζεται για αυτούς ο Manager.

Table 6.1: Αξιολόγηση τεχνολογικών επιλογών

Επιλογή	Όφελος	Trade-off
PayloadCMS v3	Rapid development, type-safety	Lock-in σε ecosystem
PostgreSQL	ACID transactions, complex queries	Setup complexity
Serverless	Zero DevOps, auto-scaling	Cold starts
Next.js App Router	SSR, Server Actions	Learning curve

6.2 Μελλοντικές Βελτιώσεις

6.2.1 Βελτίωση UX στο Admin Panel

- GUI Builder για Templates με drag-and-drop
- Visual Auto Action Editor με flow-chart style
- Wizard-style user creation

6.2.2 Επέκταση Channel Manager Support

- Hostaway (υψηλή προτεραιότητα)
- Lodgify (μεσαία προτεραιότητα)
- Custom/Direct integrations

6.2.3 Mobile Application

- Push notifications για νέες κρατήσεις
- Quick actions (approve, respond)
- Offline calendar viewing

6.2.4 Advanced Analytics Dashboard

- Revenue per property/room
- Occupancy rates over time
- Booking source distribution

- Commission breakdown trends

6.2.5 Payment Integration

Σύνδεση με payment gateways για:

- Online bill payment από Owners
- Direct booking payments στο website
- Commission auto-deduction

Chapter 7

Συμπεράσματα

7.1 Επίτευξη Στόχων

Η πλατφόρμα Lunarety επιτυγχάνει τον στόχο της να γεφυρώσει το χάσμα μεταξύ Channel Managers και ιδιοκτητών ακινήτων. Παρέχει στους Managers ένα ισχυρό εργαλείο για να προσφέρουν προστιθέμενη αξία στους πελάτες τους (Owners), ενώ παράλληλα αυτοματοποιεί χρονοβόρες διαδικασίες μέσω των Auto Actions και της Τεχνητής Νοημοσύνης.

Table 7.1: Κύρια επιτεύγματα της πλατφόρμας

Στόχος	Υλοποίηση	Αποτέλεσμα
Two-way sync με Beds24	Webhooks + API integration	Real-time data consistency
Αυτοματοποίηση ειδοποιήσεων	Auto Actions με time-based και instant triggers	80% μείωση χειροκίνητων εργασιών
Ιεραρχία χρηστών	4-level user system με granular permissions	Ανεξαρτησία διαχείρισης ανά επίπεδο
Generated websites	One-click Vercel deploy	2-λεπτά deployment time
AI integration	OpenRouter + Vercel AI SDK	24/7 ψηφιακή εξυπηρέτηση

7.2 Τεχνικά Συμπεράσματα

Αρχιτεκτονικές αποφάσεις που δικαιώθηκαν:

1. **PayloadCMS v3:** Η επιλογή του PayloadCMS ως «Backend Framework» επέτρεψε ταχεία ανάπτυξη με type-safety. Το hook system αποδείχθηκε ιδανικό για complex business logic.
2. **PostgreSQL vs MongoDB:** Η επιλογή PostgreSQL δικαιώθηκε πλήρως. Οι complex queries για billing, access control, και analytics θα ήταν δυσκολότερες με document-based storage.
3. **Serverless Architecture:** Η χρήση Vercel serverless functions απλοποίησε το deployment και μείωσε τα operation costs, με trade-off τα occasional cold starts.
4. **Monorepo Structure:** Η οργάνωση σε monorepo διευκόλυνε τη συντήρηση και την κοινή χρήση τύπων μεταξύ frontend και backend.

7.3 Επιχειρηματικό Αντίκτυπο

Η πλατφόρμα δημιουργεί νέες επιχειρηματικές ευκαιρίες:

- **Για Managers:** Προσφέρουν premium υπηρεσίες στους Owners τους, με αυτοματοποιημένη τιμολόγηση και επαγγελματικά websites
- **Για Owners:** Μειωμένη εξάρτηση από third-party OTAs (Airbnb, Booking) και χαμηλότερες προμήθειες
- **Για Staff:** Απλοποιημένη πρόσβαση σε πληροφορίες κρατήσεων χωρίς να απαιτείται τεχνική κατάρτιση

7.4 Βασικά Διδάγματα

1. **Direct DB Operations για Mass Import:** Η παράκαμψη του lifecycle για bulk operations είναι απαραίτητη για αποδεκτή απόδοση.
2. **Flag-based Loop Prevention:** Η χρήση του `skipChannelSync` flag αποδείχθηκε απλή και αποτελεσματική λύση για τον two-way sync.
3. **Feature-based Access Control:** Η ευελιξία του συστήματος δικαιωμάτων επιτρέπει κάθε Manager να προσαρμόζει την εμπειρία για τους χρήστες του.
4. **AI as Enhancement:** Η AI ενσωμάτωση λειτουργεί καλύτερα ως «enhancement» παρά ως «replacement» – βελτιώνει την εμπειρία χωρίς να αντικαθιστά ανθρώπινη κρίση.

Η αρχιτεκτονική της πλατφόρμας επιτρέπει εύκολη επέκταση και συντήρηση, καθιστώντας την μια βιώσιμη λύση για τη σύγχρονη αγορά βραχυχρόνιας μίσθωσης.

Βιβλιογραφία

- [1] Boris Cherny. *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media, 2019. isbn: 978-1492037651.
- [2] Eric Jonas et al. “Cloud Programming Simplified: A Berkeley View on Serverless Computing.” In: *Technical Report No. UCB/EECS-2019-3* (2019).
- [3] Vercel. *Next.js Documentation*. 2024. url: <https://nextjs.org/docs> (visited on 11/01/2024).
- [4] PayloadCMS. *PayloadCMS Documentation*. 2024. url: <https://payloadcms.com/docs> (visited on 11/01/2024).
- [5] Drizzle Team. *Drizzle ORM Documentation*. 2024. url: <https://orm.drizzle.team/> (visited on 11/01/2024).
- [6] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. url: <https://www.postgresql.org/docs/> (visited on 11/01/2024).
- [7] shadcn. *Shadcn UI Documentation*. 2024. url: <https://ui.shadcn.com/> (visited on 11/01/2024).
- [8] Alex Banks and Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2nd. O'Reilly Media, 2020. isbn: 978-1492051725.
- [9] pmndrs. *Zustand Documentation*. 2024. url: <https://docs.pmnd.rs/zustand> (visited on 11/01/2024).
- [10] OpenAPI Initiative. *OpenAPI Specification*. 2024. url: <https://spec.openapis.org/oas/latest.html> (visited on 11/01/2024).
- [11] Tom Brown et al. “Language Models are Few-Shot Learners.” In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.
- [12] Vercel. *Vercel AI SDK Documentation*. 2024. url: <https://sdk.vercel.ai/docs> (visited on 11/01/2024).
- [13] OpenRouter. *OpenRouter Documentation*. 2024. url: <https://openrouter.ai/docs> (visited on 11/01/2024).
- [14] Rob Law, Daniel Leung, and Ivan Chan. “Progress and Development of Information Technology in the Hospitality Industry: Evidence from Cornell Hospitality Quarterly.” In: *Cornell Hospitality Quarterly* 61.1 (2020), pp. 19–45.
- [15] Beds24. *Beds24 API Documentation v2.0*. 2024. url: <https://beds24.com/api/v2/> (visited on 11/01/2024).

-
- [16] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. isbn: 978-0201633610.
 - [17] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. isbn: 978-1492043454.

Appendix A

Δομή Webhook Payload

Listing A.1: Παράδειγμα Webhook Payload από Beds24

```
1 {  
2   "booking": {  
3     "id": 12345,  
4     "arrival": "2025-01-01",  
5     "departure": "2025-01-05",  
6     "status": "confirmed"  
7   },  
8   "messages": [...]  
9 }
```

Appendix B

Παράδειγμα Auto Action Query

Listing B.1: Αναζήτηση κρατήσεων για before-checkin trigger

```
1  const whereClause = {  
2    'resource.checkInStartOfDay': {  
3      greater_than_equal: fromDate.toISOString(),  
4      less_than_equal: toDate.toISOString(),  
5    },  
6    'logging.automationTracking.triggeredAutomations': {  
7      not_in: [autoAction.id],  
8    }  
9  }
```